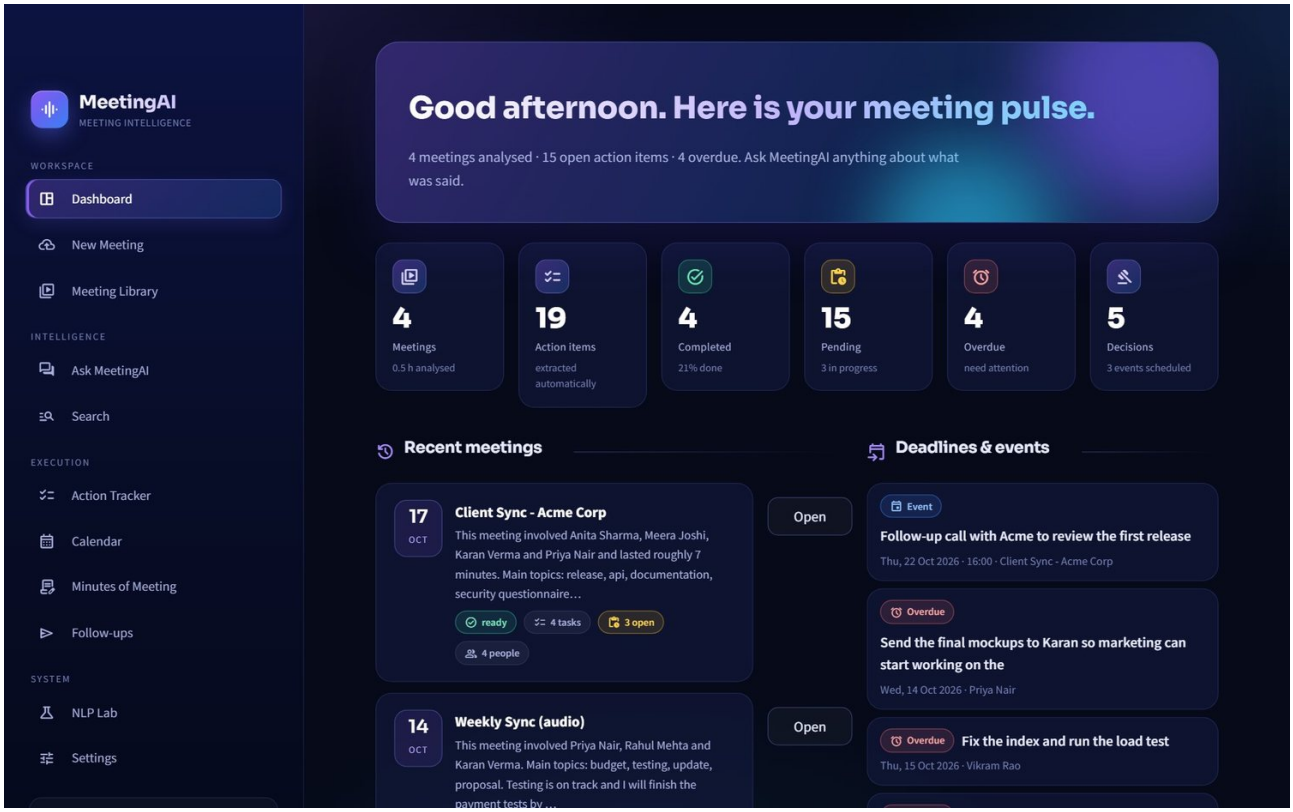
MeetingAI - Project Report

AI-powered meeting intelligence: transcripts, summaries, decisions, action items, calendar, chat and minutes - with names on every voice.

Live application	https://gd4n8x8sjkkxmdhfptkc9m.streamlit.app/
Source code	https://github.com/nidhitiwari23/Meeting-AI-
Built with	Python, Streamlit, SQLite (FTS5), ChromaDB, scikit-learn, Whisper, pyannote, free LLMs (Groq / Gemini / OpenRouter)
Companion document	*MeetingAI - Code Walkthrough* (every file explained)



The MeetingAI dashboard

Contents

1 Executive summary
2 The problem and the objectives
3 A tour of the application (screenshots)
4 System architecture
5 End-to-end processing flow
6 Speech and "who said what"
7 Classical NLP: how sentences are understood
8 The confidence gateway
9 The LLM layer and the validation layer
10 Search and the chatbot
11 Minutes, calendar, follow-ups and e-mail
12 Data model and storage
13 User-interface design
14 Technology stack and why each piece was chosen
15 Security, privacy and honesty by design
16 Deployment
17 Testing and evaluation
18 Limitations and future work
19 Source-code map (one line per file)
20 Glossary

---

1. Executive summary

Meetings produce decisions, promises and deadlines - and most of them are forgotten. **MeetingAI** turns a meeting recording or transcript into organised, searchable knowledge:

- a **transcript** with timestamps and **speaker names** ("Priya: ...", not "Speaker 2"),
- a structured **summary** (overview, key points, decisions, conclusions),
- **action items** with owner, deadline, priority and status - or "Unassigned" / "Not specified" when the meeting did not say,
- **decisions** and **dated events** that are placed on a **calendar** automatically,
- a **chatbot** and a **cross-meeting search** whose answers come only from the meetings and always show their sources,
- editable **Minutes of Meeting** (Word, PDF, Markdown) that can be **e-mailed** after confirmation,
- **follow-up suggestions** (recaps, reminders) that are only ever drafts until you confirm.

What makes the project different from "send the transcript to ChatGPT":

1 **Hybrid NLP.** Fast, explainable classical NLP runs first; a **confidence gateway** sends only doubtful sentences to a free LLM.

2. **Validation.** Everything an LLM returns is checked against the real transcript (verbatim quotes, real owners, dates computed by code).
3. **Honesty.** When something is unknown, the application says so instead of inventing it.
4. **Runs free.** Free-tier LLMs, a local embedding model, SQLite and ChromaDB - deployable on Streamlit Community Cloud.

2. The problem and the objectives

Problem. Important information is buried in long conversations. Manual minutes are slow, normal transcription gives a wall of text, and generic chatbots can invent facts.

Objectives (all met):

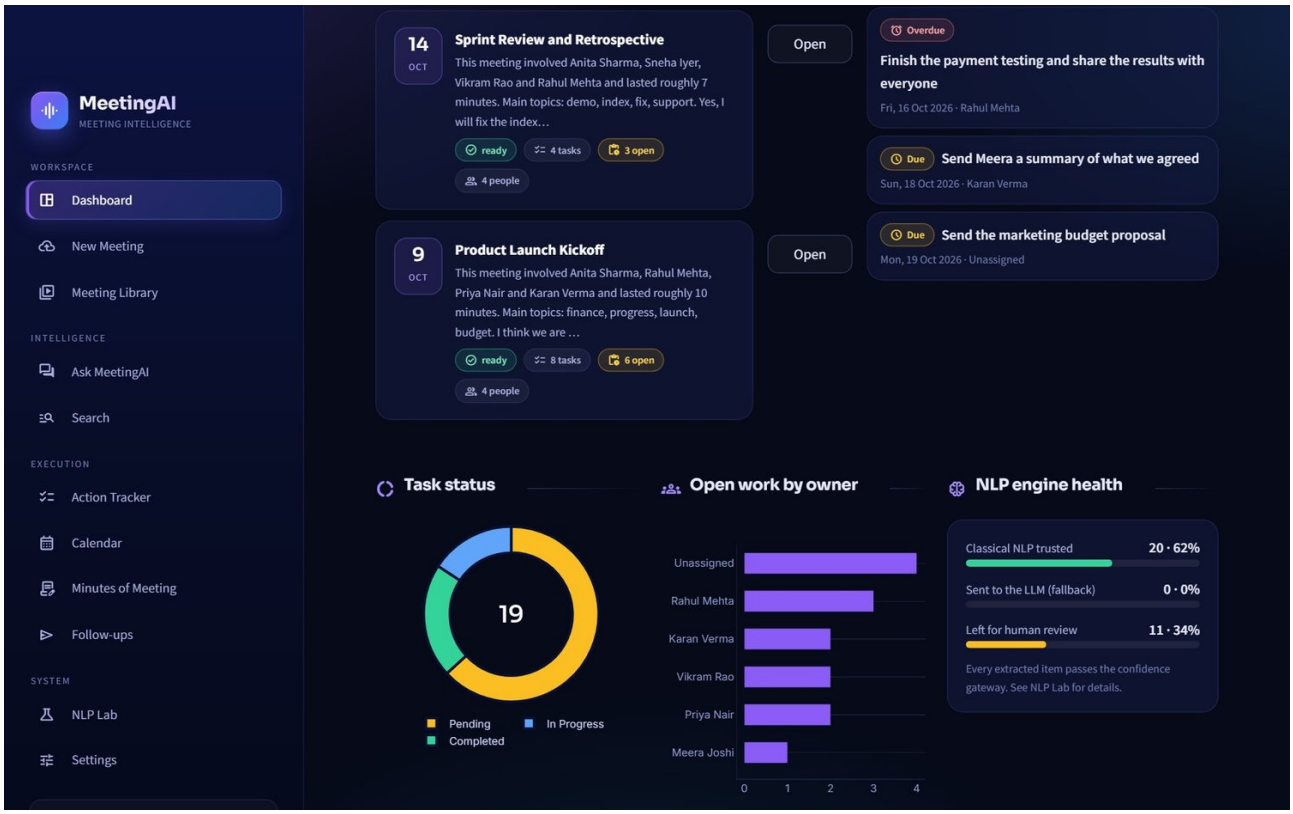
#	Objective	Where it lives
1	Upload audio, video or text transcripts	New Meeting page, `parsing.py`, `stt.py`
2	Speech-to-text with speakers and timestamps	`stt.py`, `diarize.py`, `speakers.py`
3	Structured AI summary	`llm_tasks.py`, `nlp.py` (offline fallback)
4	Action items with owner / deadline / status, editable	`nlp.py`, `gateway.py`, Action Tracker
5	Grounded meeting chatbot	`rag.py`, Ask MeetingAI page
6	Cross-meeting search with sources	`rag.py`, `db.py` (FTS5), Search page
7	Meeting library and history (incl. chat history)	`db.py`, Library page
8	Minutes of Meeting: edit, accept, export, e-mail	`mom.py`, `mailer.py`
9	Events and tasks on a calendar	`dates.py`, Calendar page, `ics.py`
10	Central action tracker	Action Tracker page
11	Follow-up suggestions with confirmation	`followups.py`
12	Dashboard statistics	Dashboard page
13	Honest handling of missing information	`gateway.py`, validation in `pipeline.py`
14	Professional dark SaaS-style UI	`ui/theme.py`, `ui/components.py`

3. A tour of the application

The left sidebar is the single navigation. Every page shares the same design language: deep-navy background, violet-blue gradients, glass cards, status chips and icons.

3.1 Dashboard

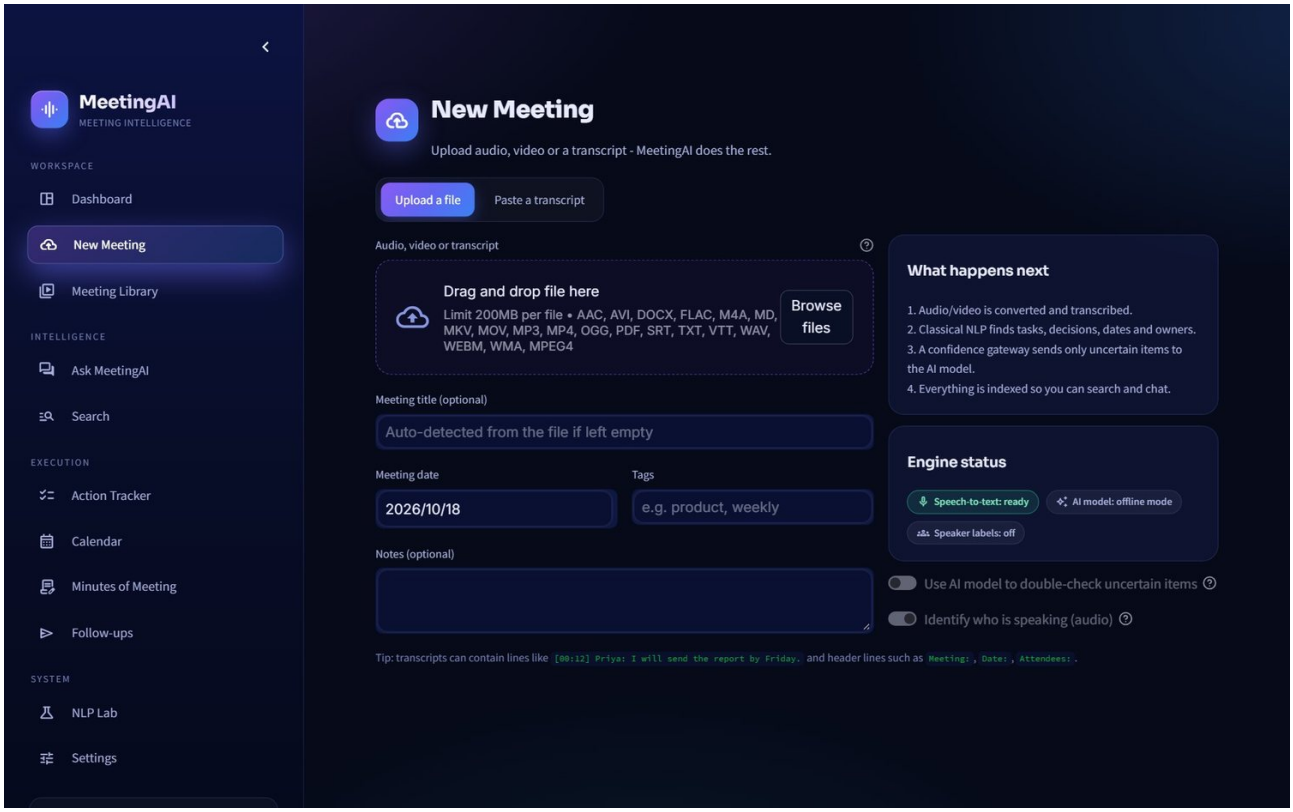
KPIs (meetings, action items, completed, pending, overdue, decisions), recent meetings, upcoming deadlines and events, a task-status donut, open work by owner, and **NLP-engine health** - how many items classical NLP handled alone versus how many went to the LLM.



Dashboard - charts and NLP engine health

3.2 New Meeting

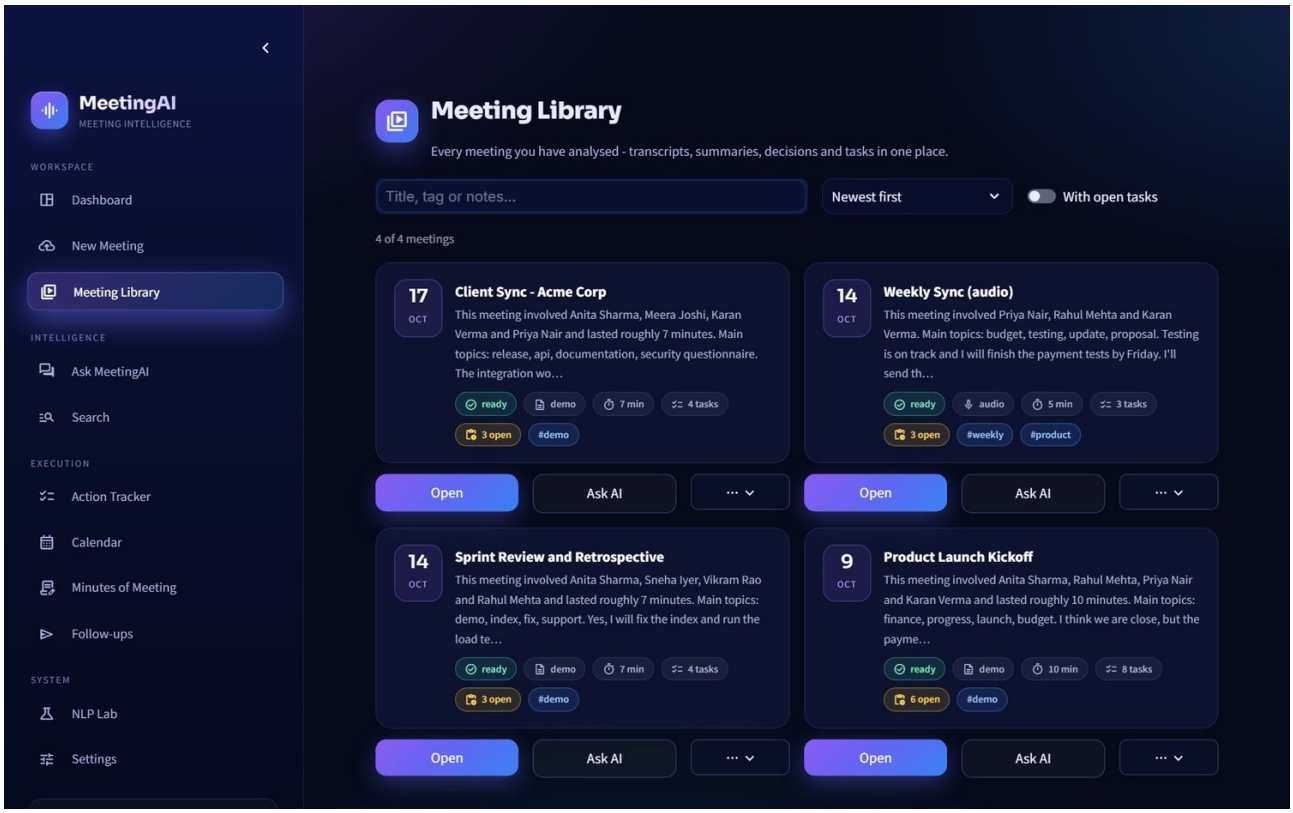
Drag in audio, video or a transcript (txt, vtt, srt, docx, pdf), or paste one. The status panel shows which engines are connected (speech-to-text, AI model, speaker labels, saved voices). Progress is shown as a live stepper.



Upload page

3.3 Meeting Library

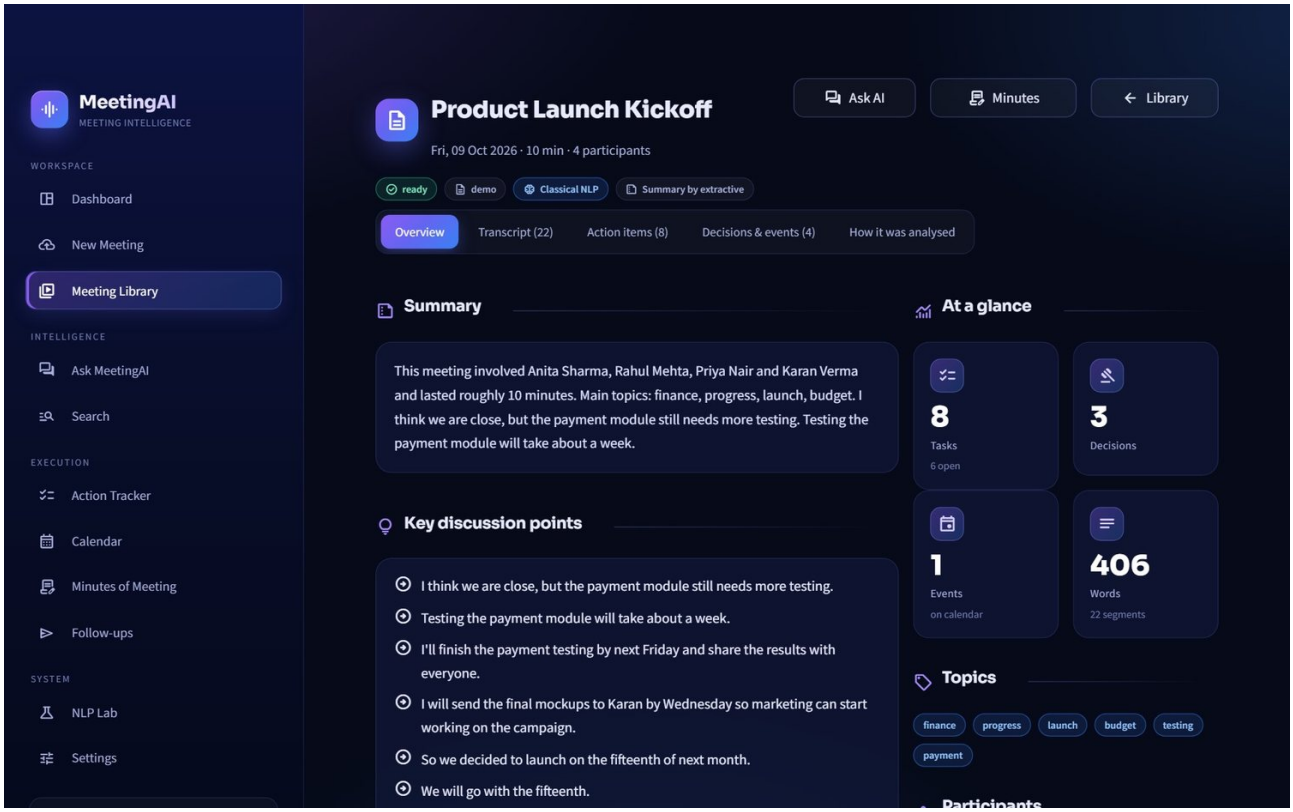
Every analysed meeting as a card: date, summary snippet, status, task counts, tags. Search, sort, open, ask AI, or delete.



Meeting library

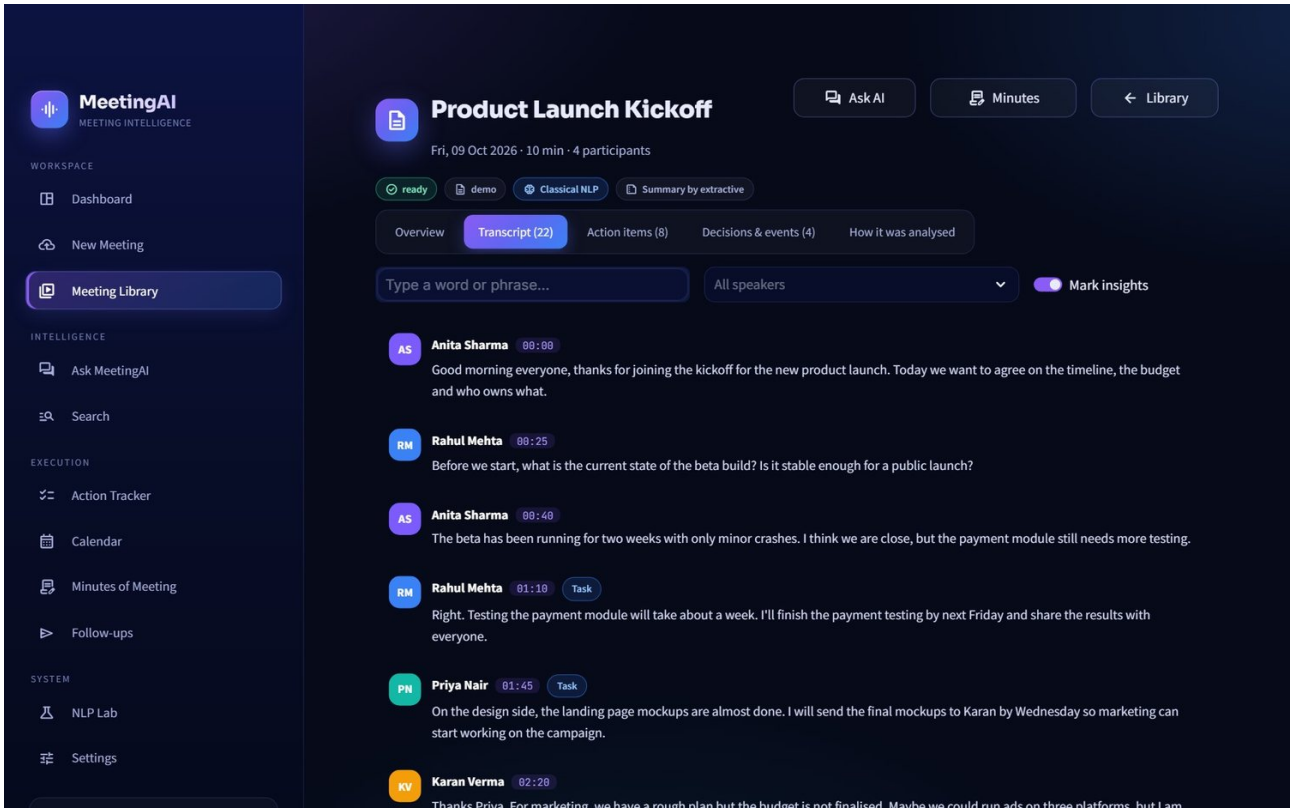
3.4 Meeting details

Overview - summary, key discussion points, conclusions, topics, participants and an "at a glance" panel.



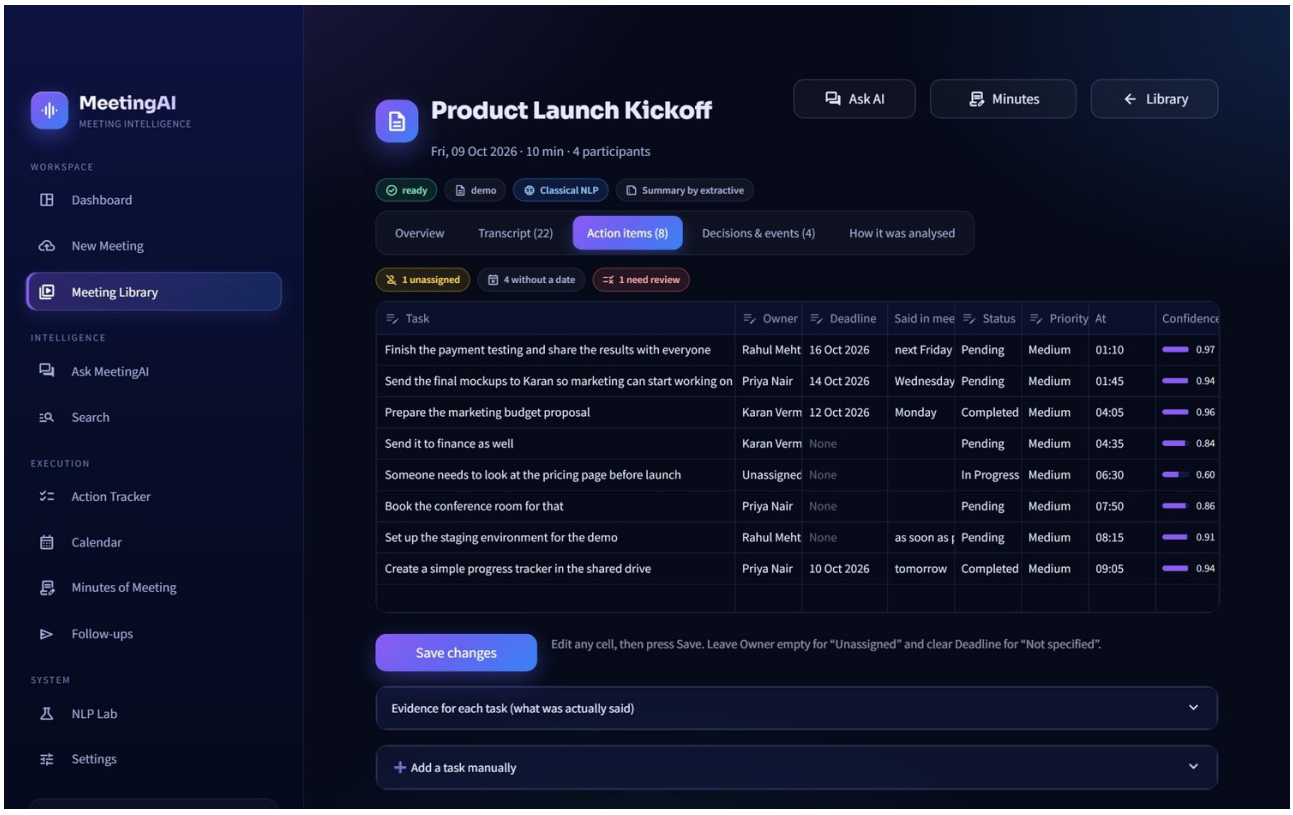
Meeting overview

Transcript - one coloured avatar per speaker, timestamps, search, speaker filter, and chips marking where tasks, decisions and events were said. If the original recording exists you can jump the audio player to any line.



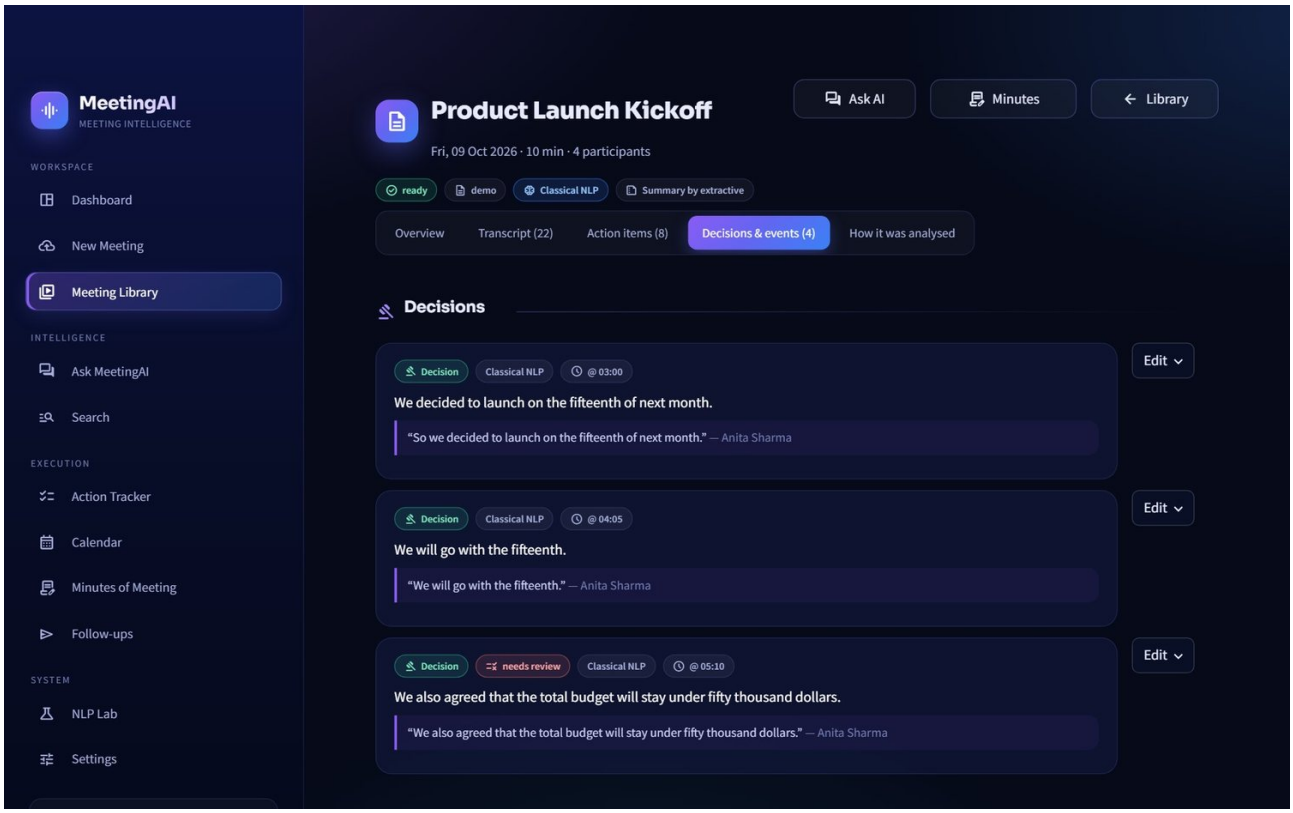
Transcript with speaker names and insight chips

Action items - an editable table (task, owner, deadline, status, priority) with the exact deadline wording, source meeting, confidence and "needs review" flags.



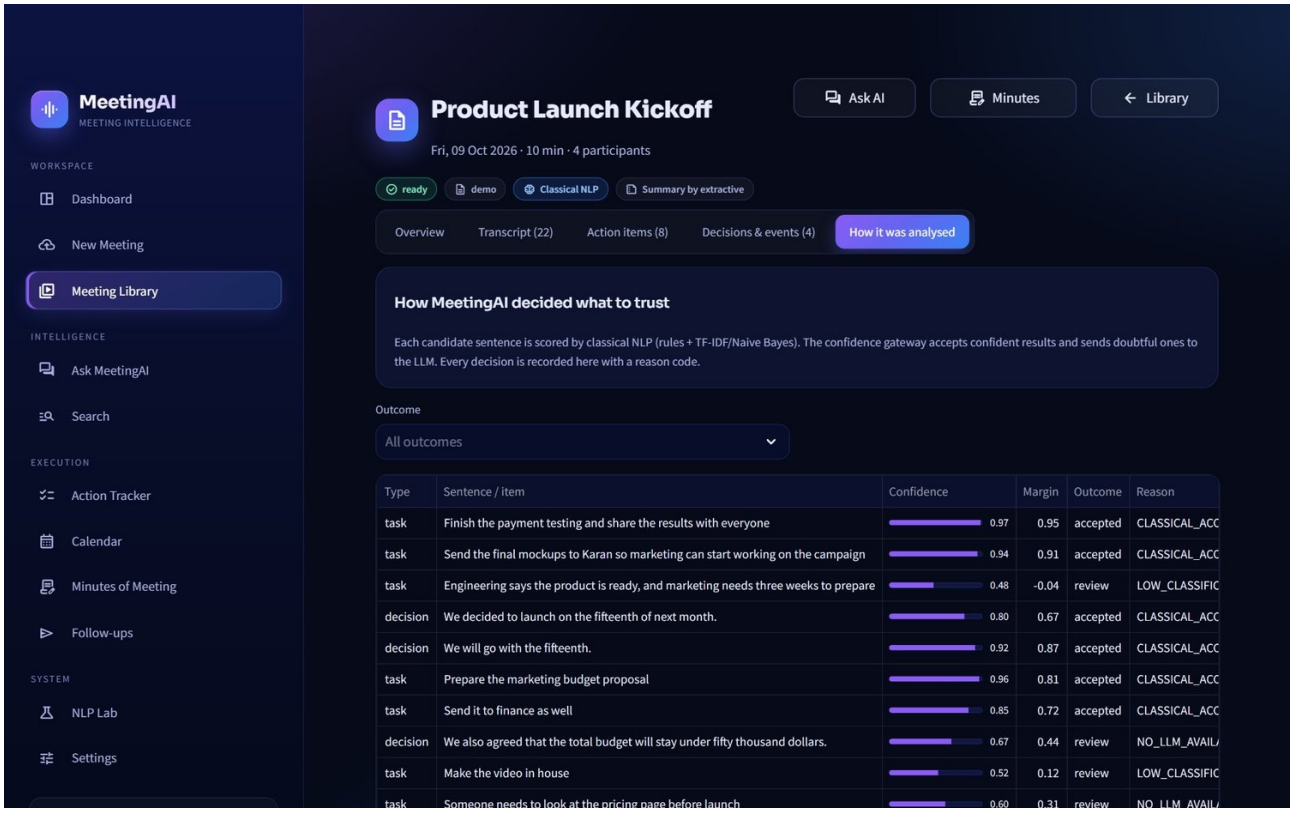
Editable action items

Decisions and events - each decision shows the quote that proves it; each event can be added to Google Calendar or downloaded as `.ics`.



Decisions and events

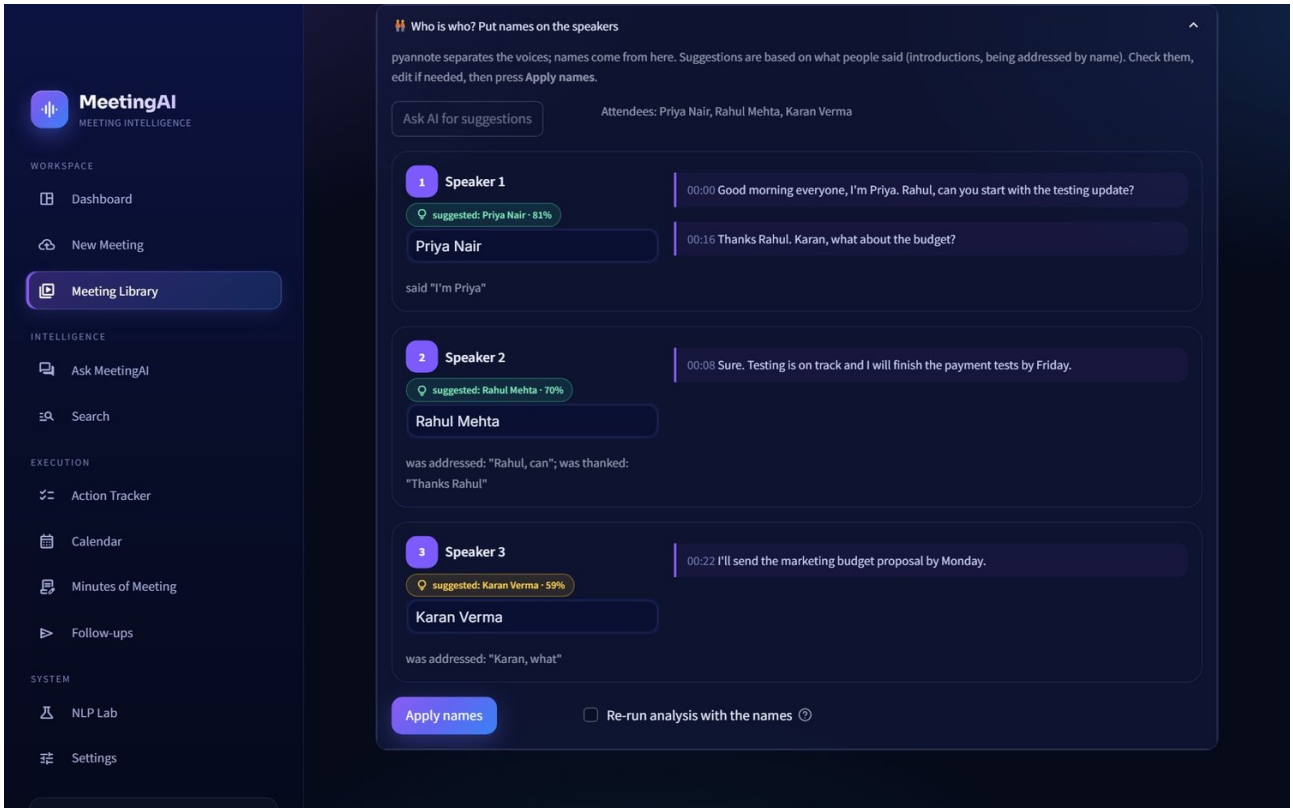
How it was analysed - the transparency tab: for every candidate sentence, the confidence, margin, outcome (accepted / sent to LLM / review), the reason code and a plain-English explanation.



Gateway decisions per meeting

3.5 Who is who? (speaker names)

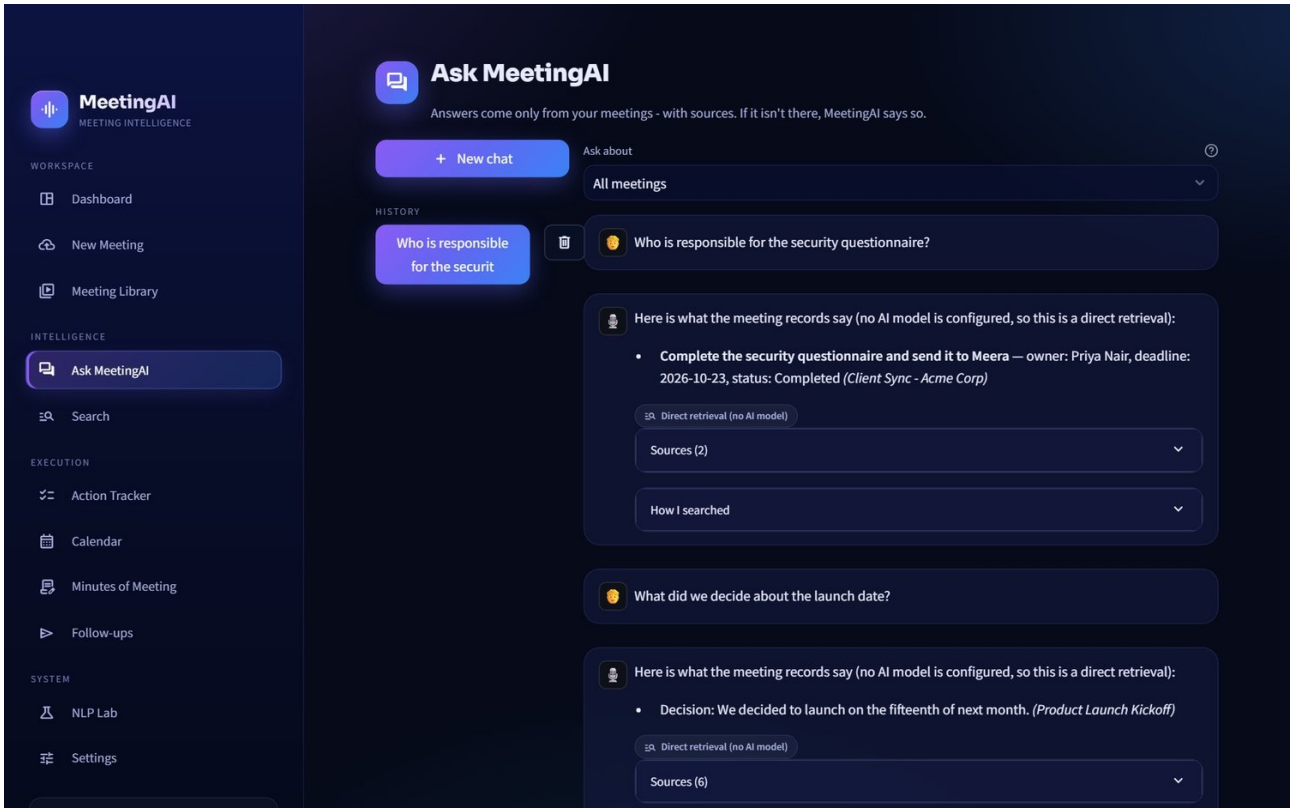
pyannote separates voices; this panel puts names on them. Suggestions come from what people said ("I'm Priya", "Rahul, can you..."), each with its evidence and confidence. Play a sample, correct the name, press **Apply names**, and optionally **Remember this voice** so future meetings label it automatically.



Who is who panel

3.6 Ask MeetingAI

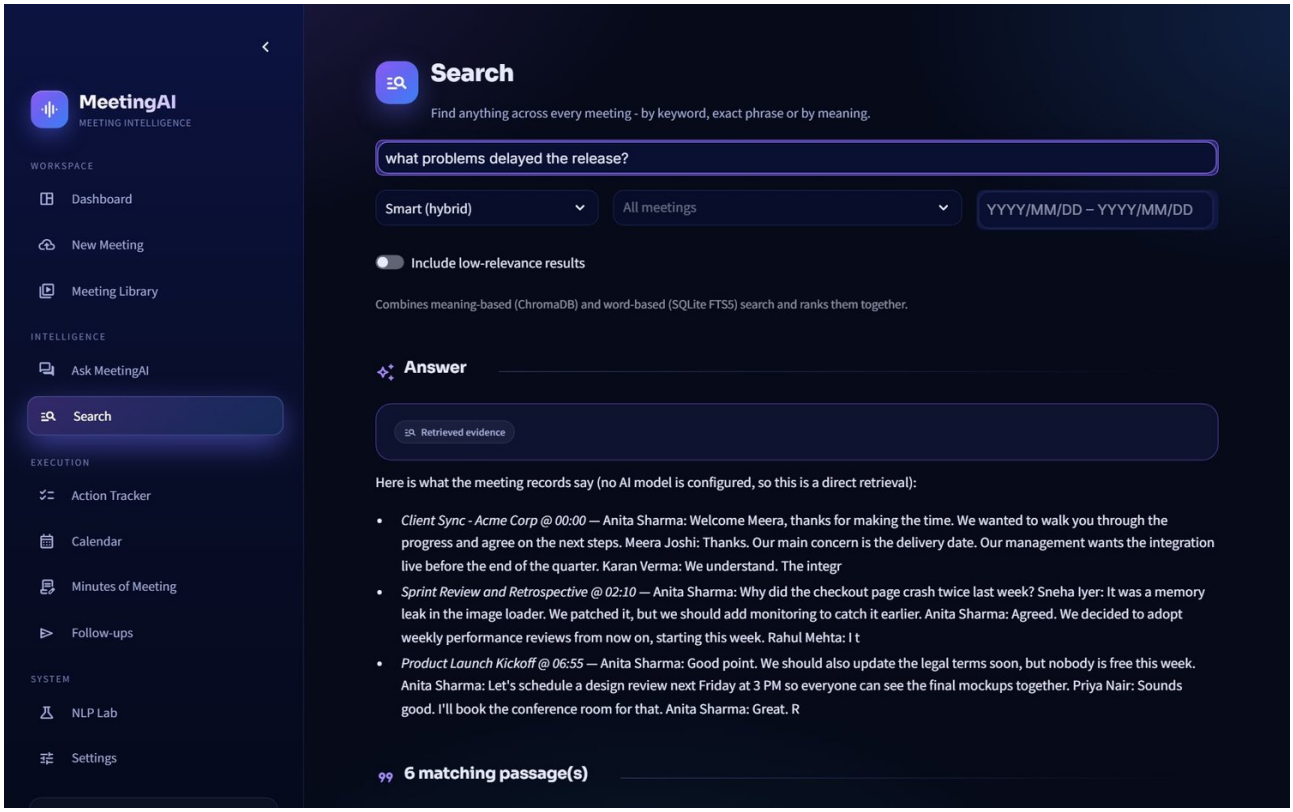
A chatbot restricted to your meetings. Answers cite sources, show *how I searched*, and say "not found" when the meetings do not contain the answer. Chats are saved and can be continued.



Chat

3.7 Search

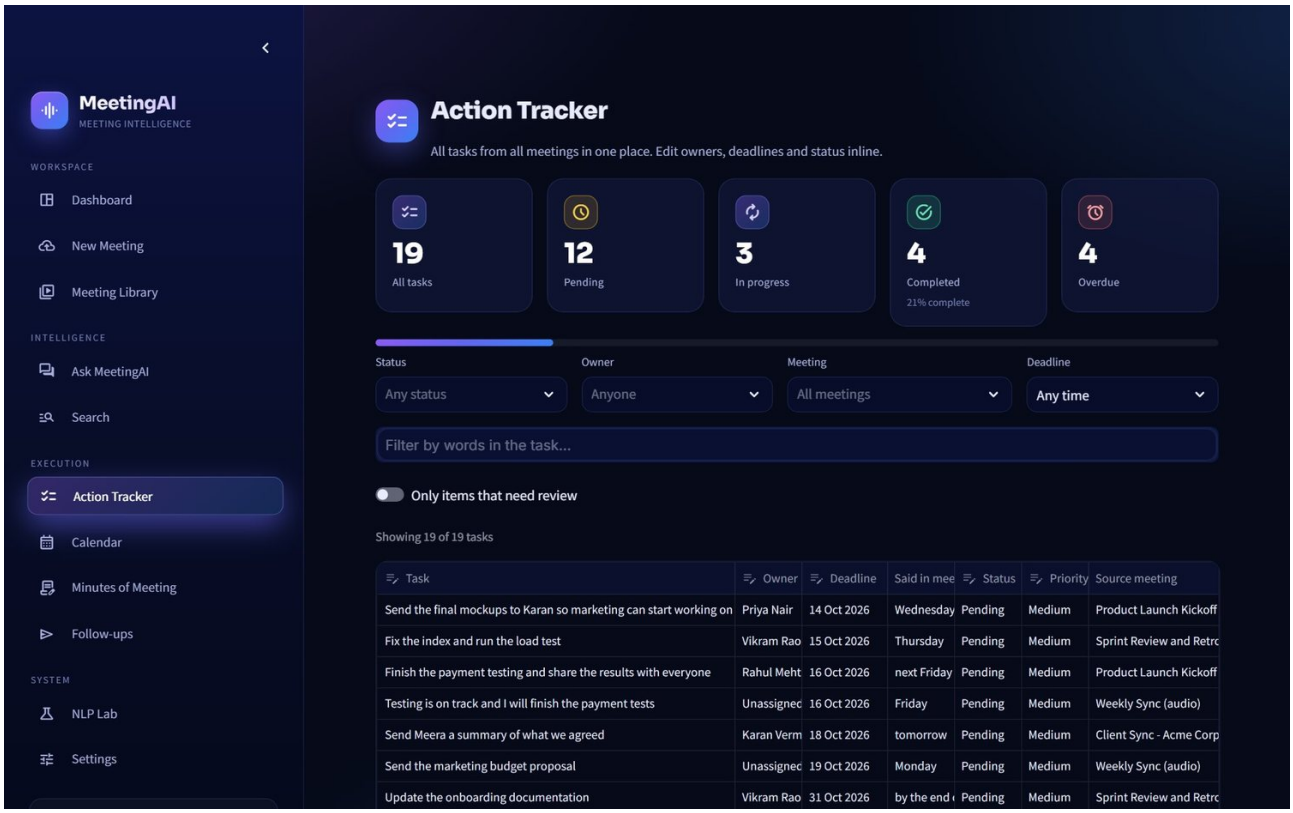
Smart (hybrid), keyword, semantic or exact-phrase search across all meetings. Results show the meeting, timestamp, speakers, the supporting text with highlights, and whether a hit came from keywords, meaning or both.



Cross-meeting search

3.8 Action Tracker

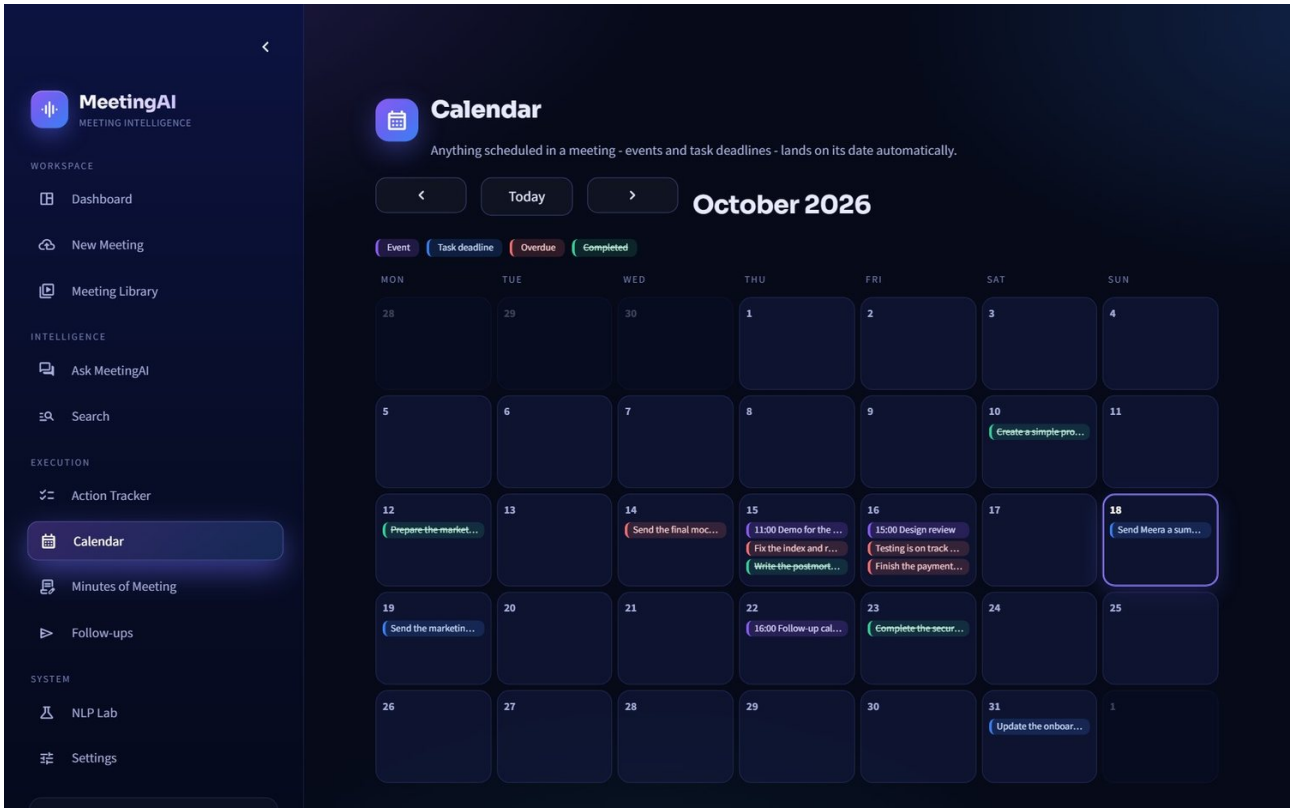
All tasks from all meetings in one table with filters (status, owner, meeting, deadline, text, needs-review). Edit inline and press *Save changes*.



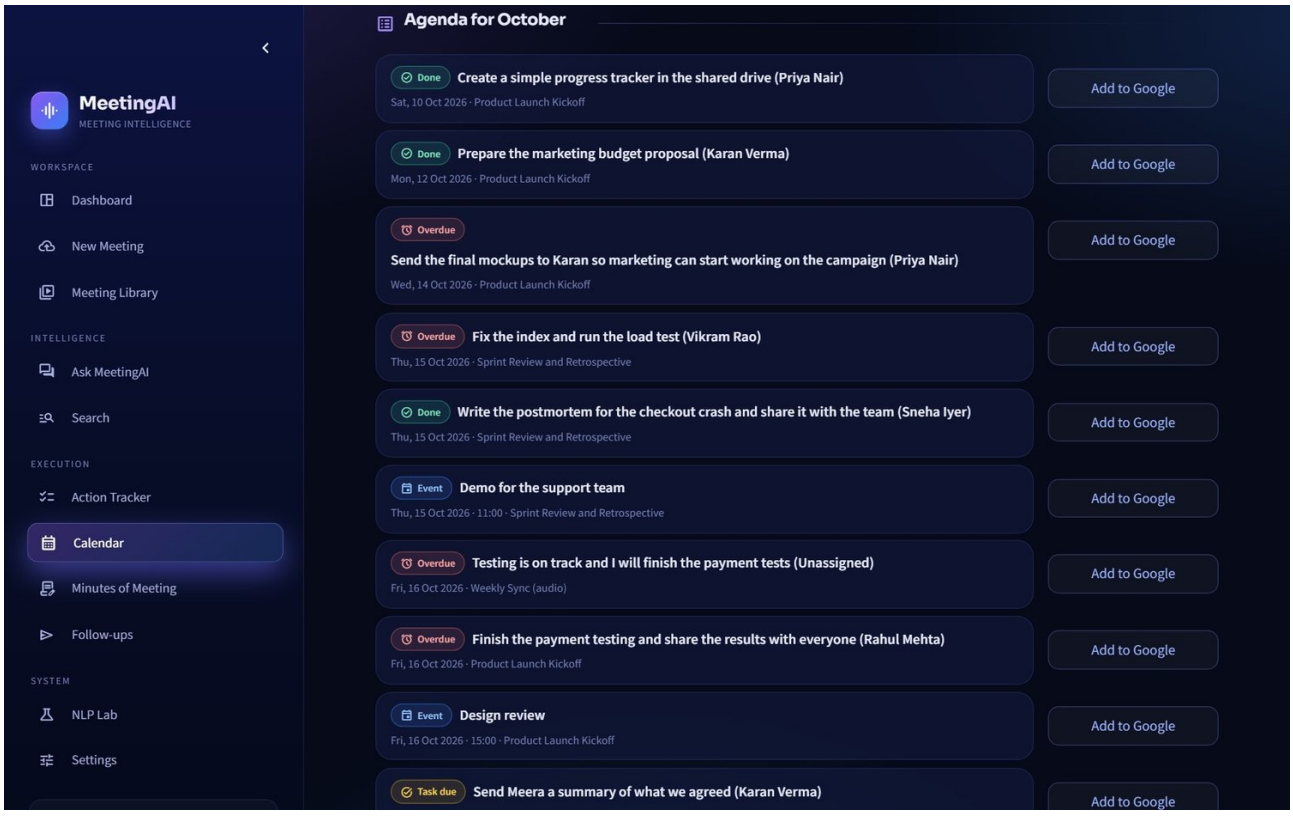
Action tracker

3.9 Calendar

A month grid. Events and task deadlines found in meetings appear on their dates (violet = event, blue = deadline, red = overdue, green = done), followed by an agenda list with *Add to Google* buttons and a downloadable `.ics` file.



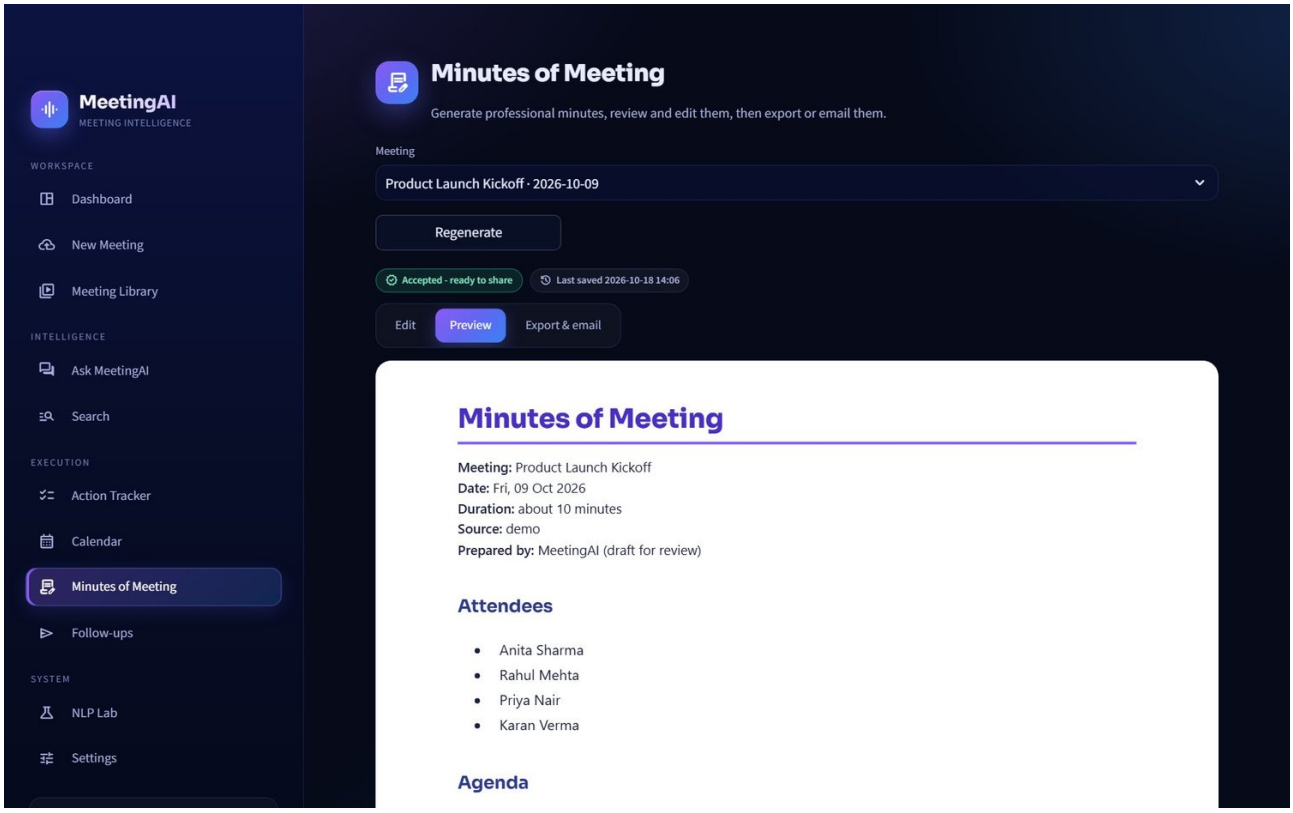
Calendar month view



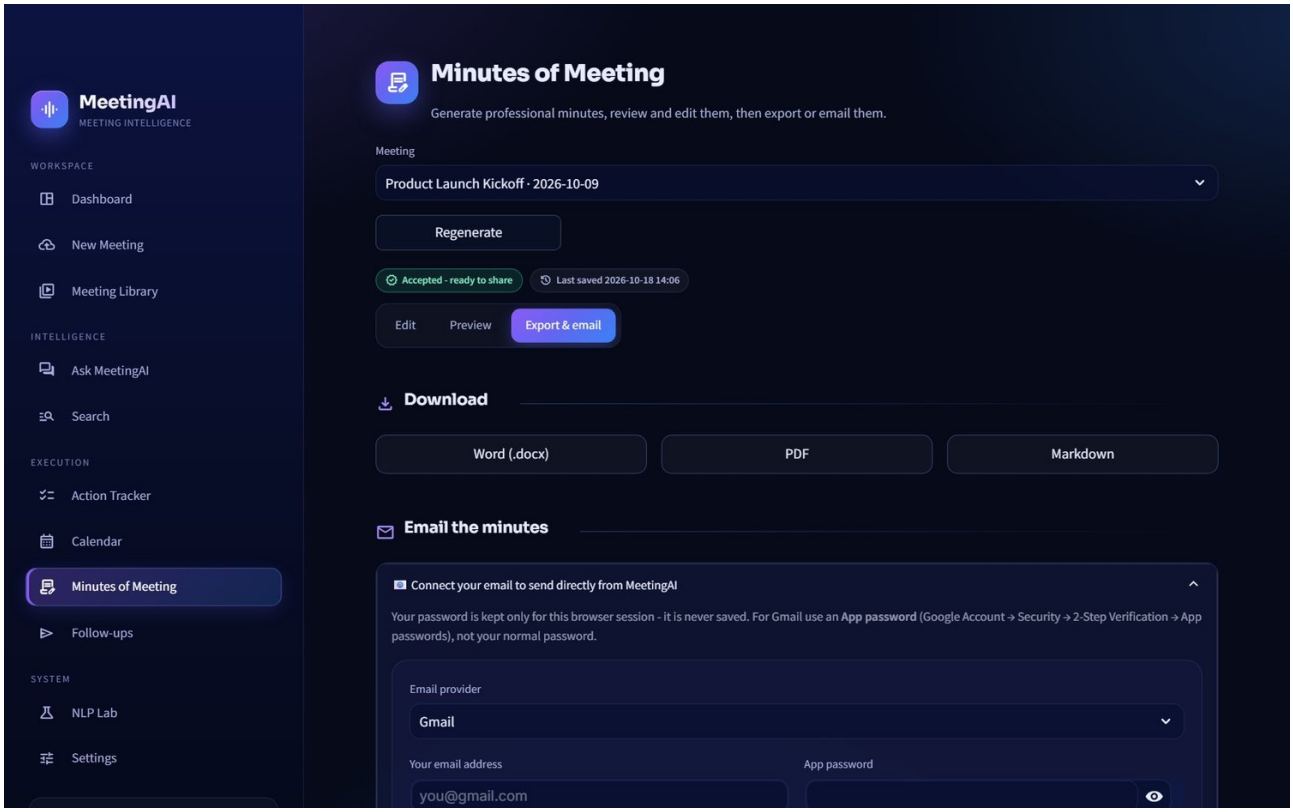
Calendar agenda

3.10 Minutes of Meeting

Generate a professional draft, edit it, preview it, **accept** it, then download Word / PDF / Markdown or e-mail it to the addresses you enter - after a confirmation tick.



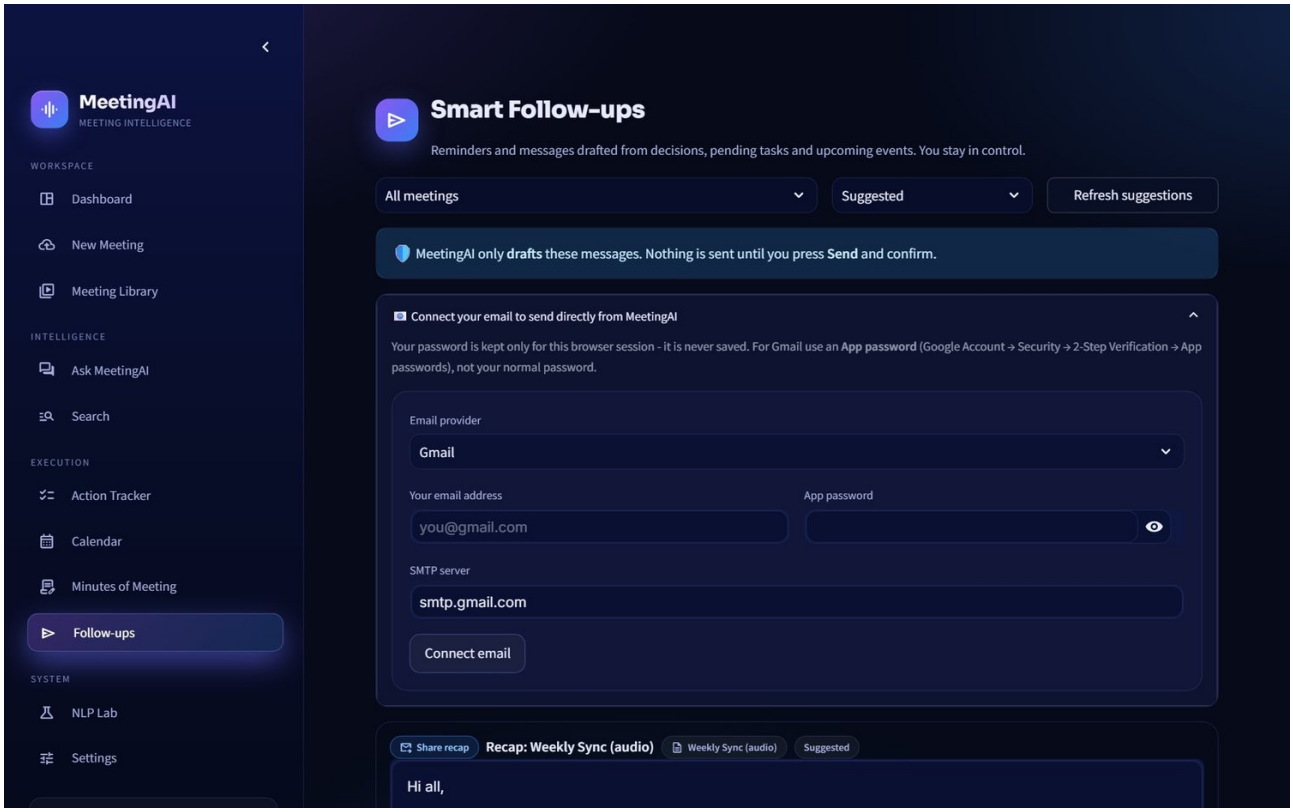
Minutes preview



Minutes export and e-mail

3.11 Follow-ups

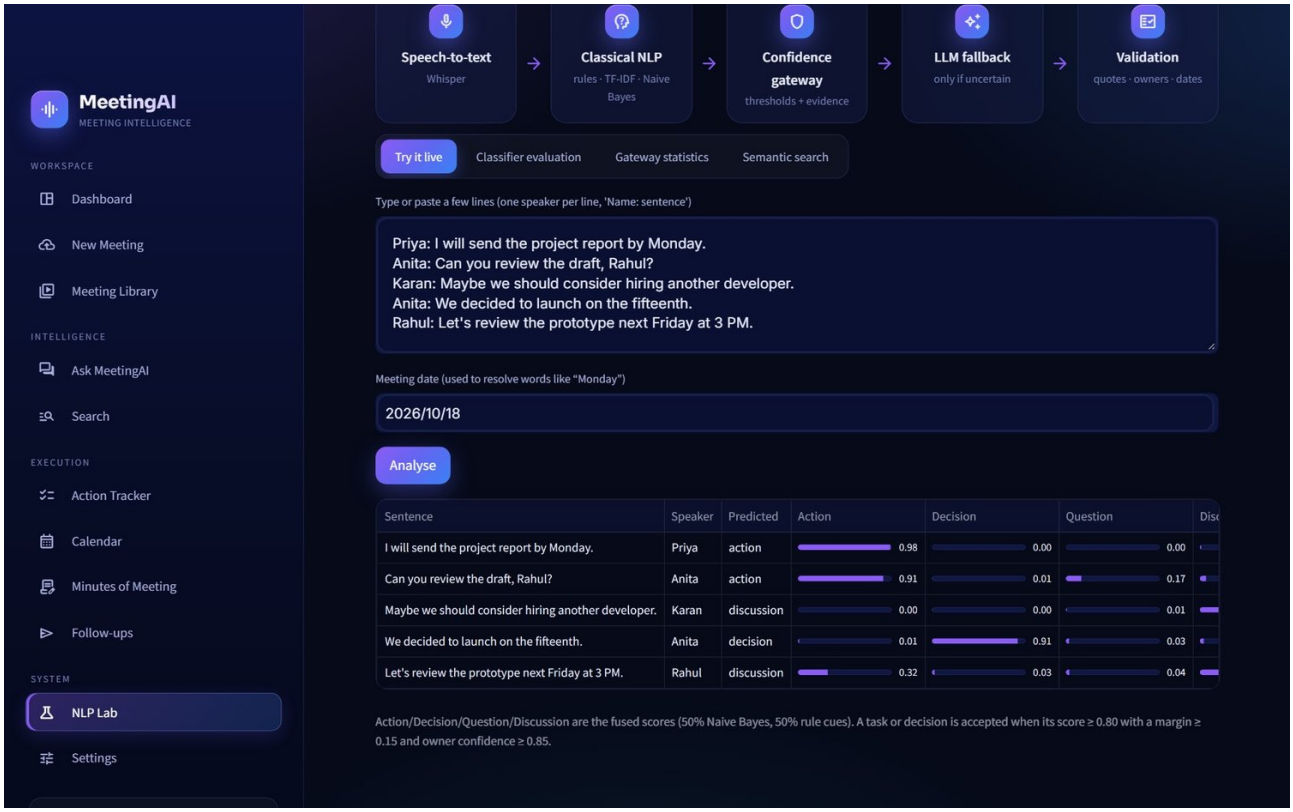
Suggested recaps, reminders per person, requests for owners of unassigned tasks and event reminders. Edit, polish with AI (facts unchanged), then send or open in your mail app.



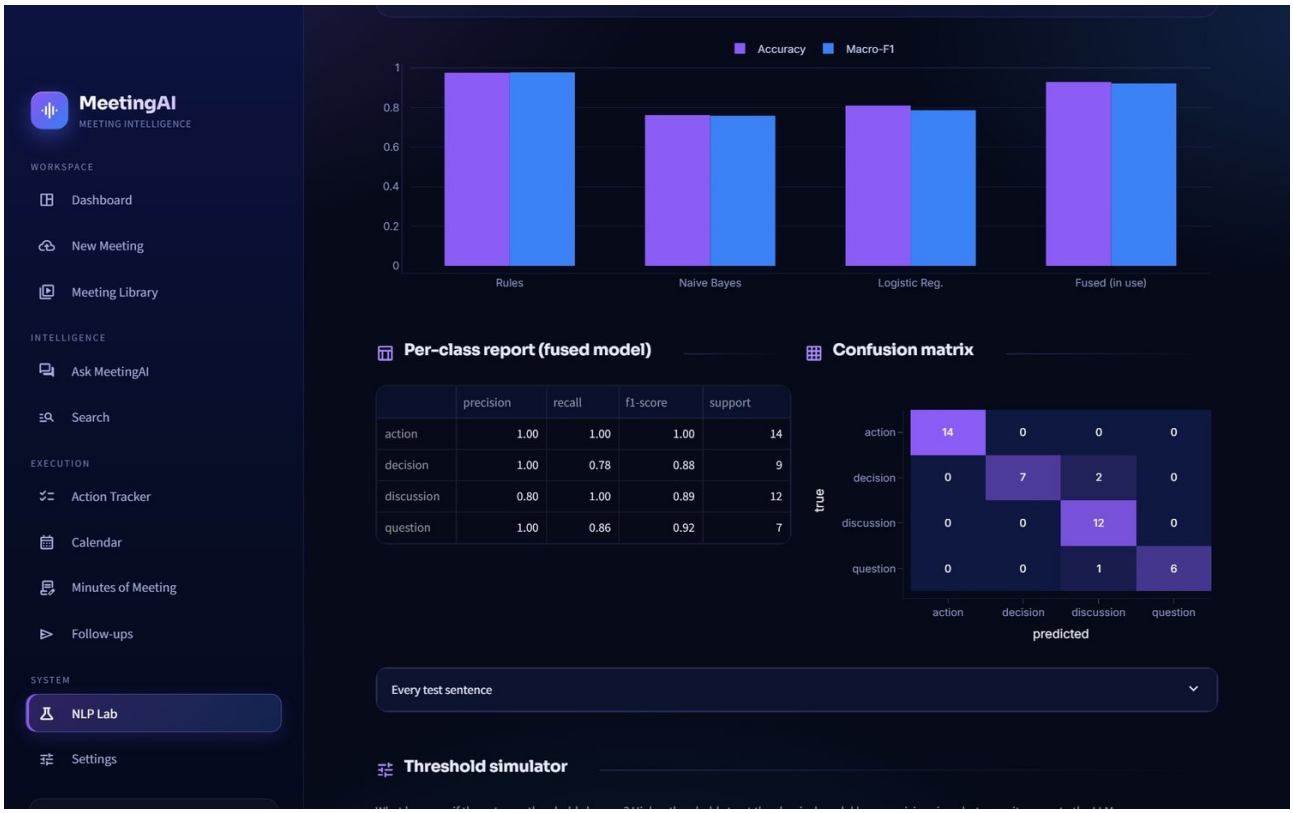
Follow-ups

3.12 NLP Lab

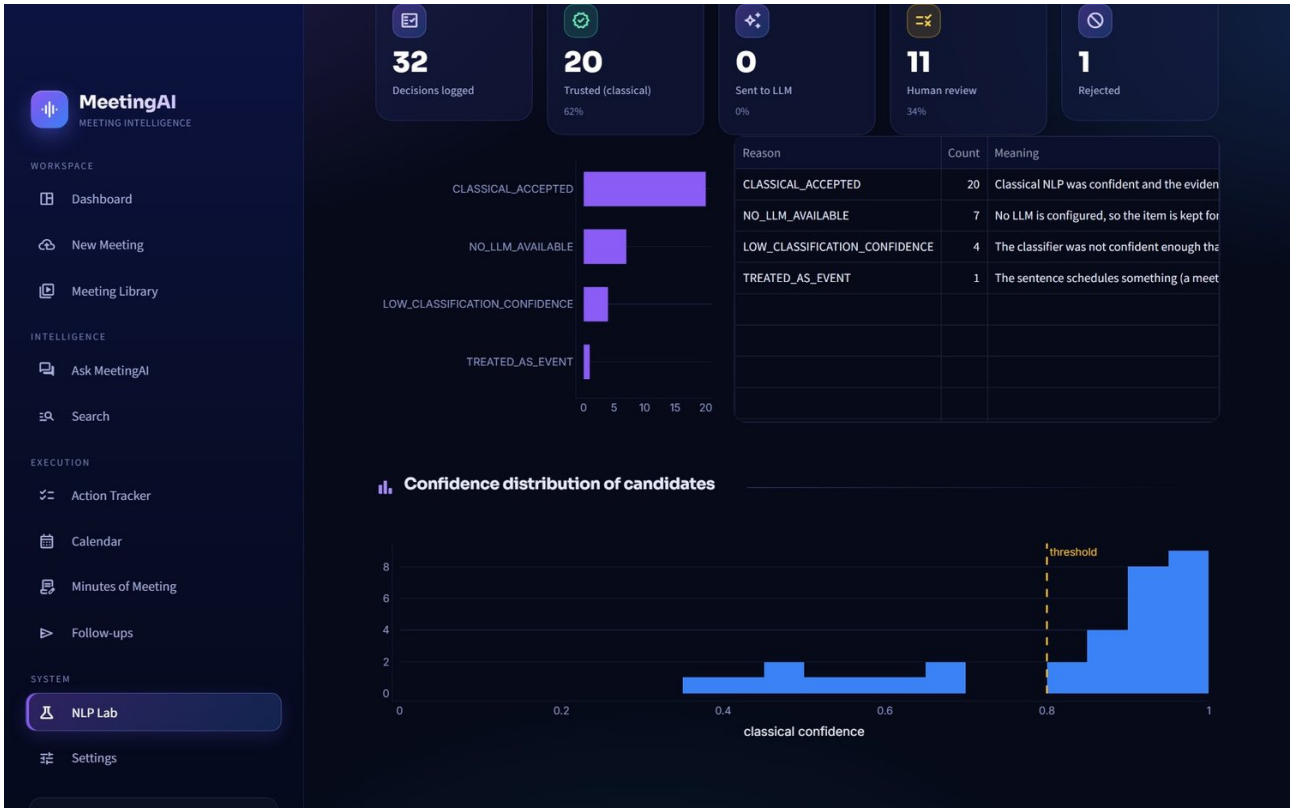
A "look under the hood" page: try the NLP live on any text, see held-out accuracy and the confusion matrix, tune the thresholds with a simulator, and see gateway statistics.



NLP Lab - live analysis



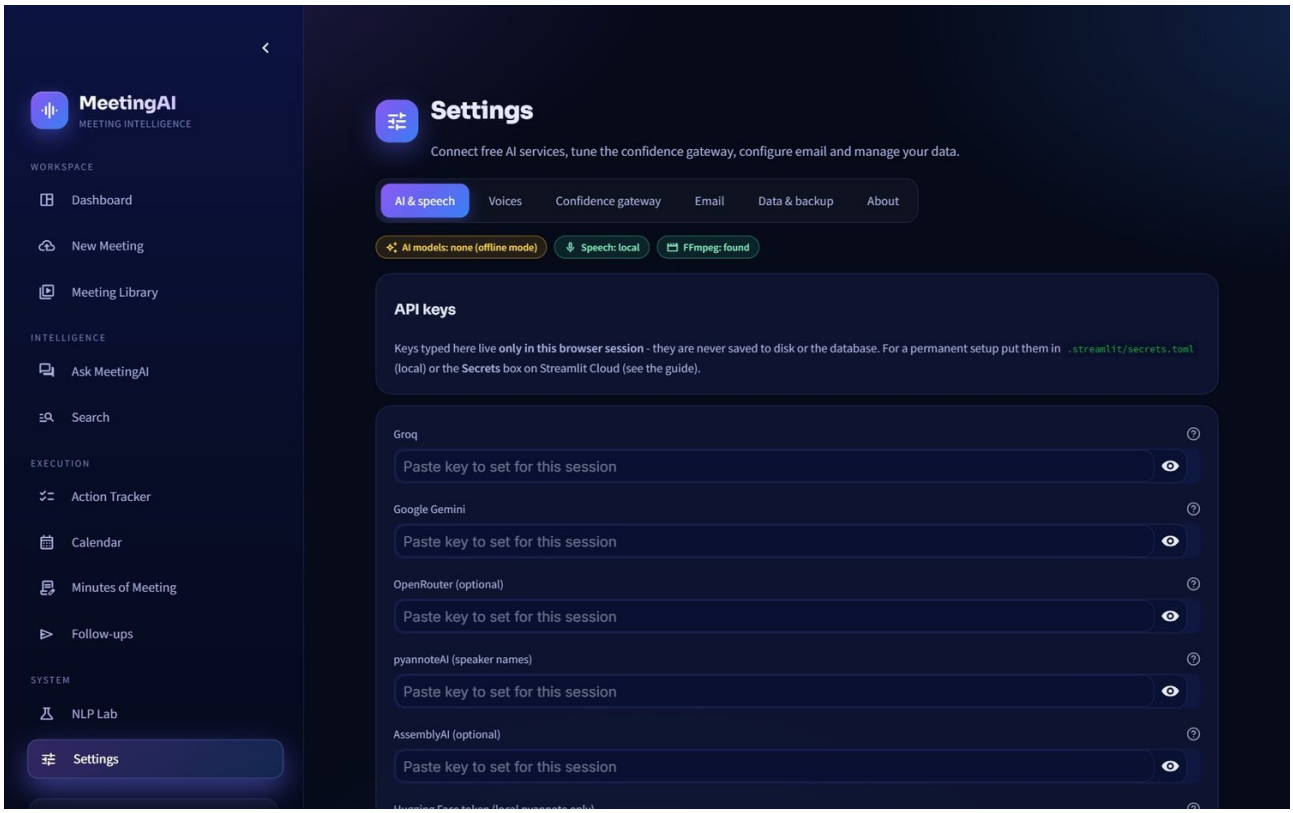
NLP Lab - classifier evaluation



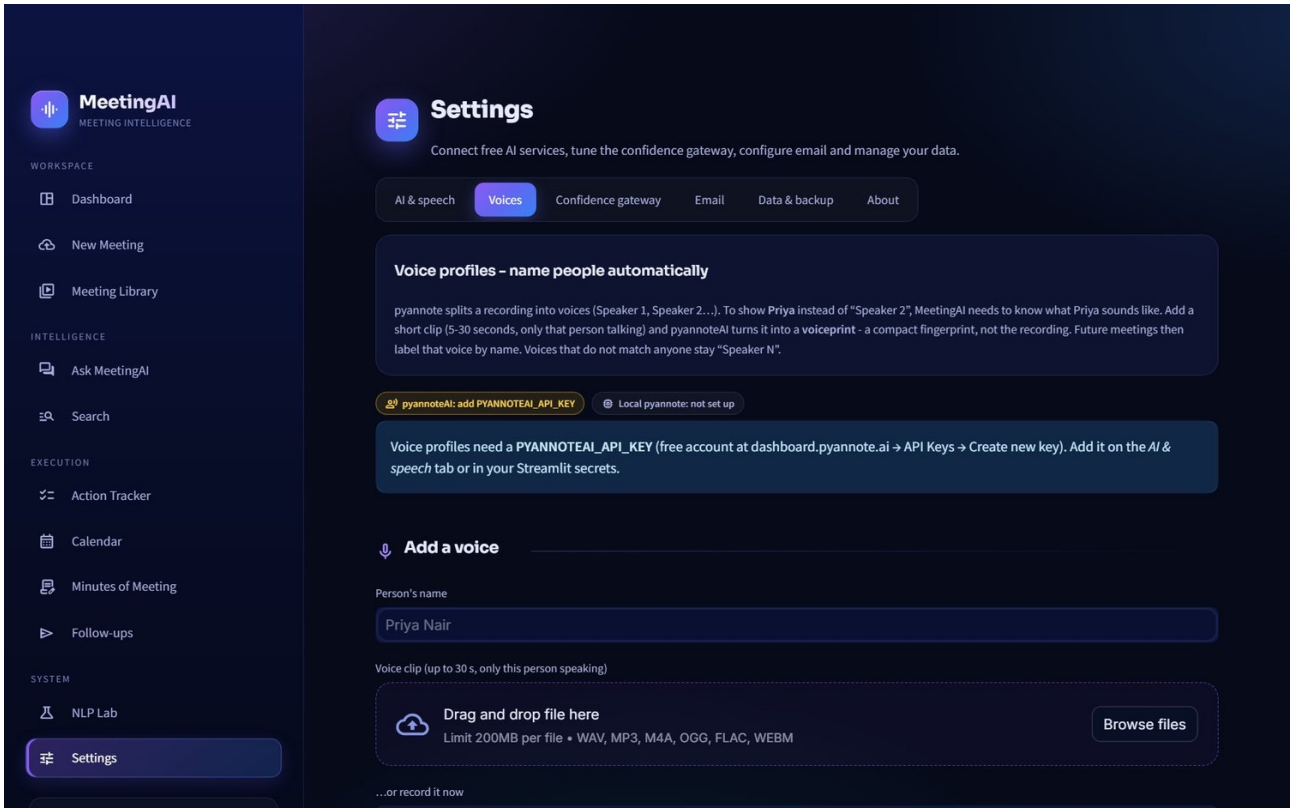
NLP Lab - gateway statistics

3.13 Settings

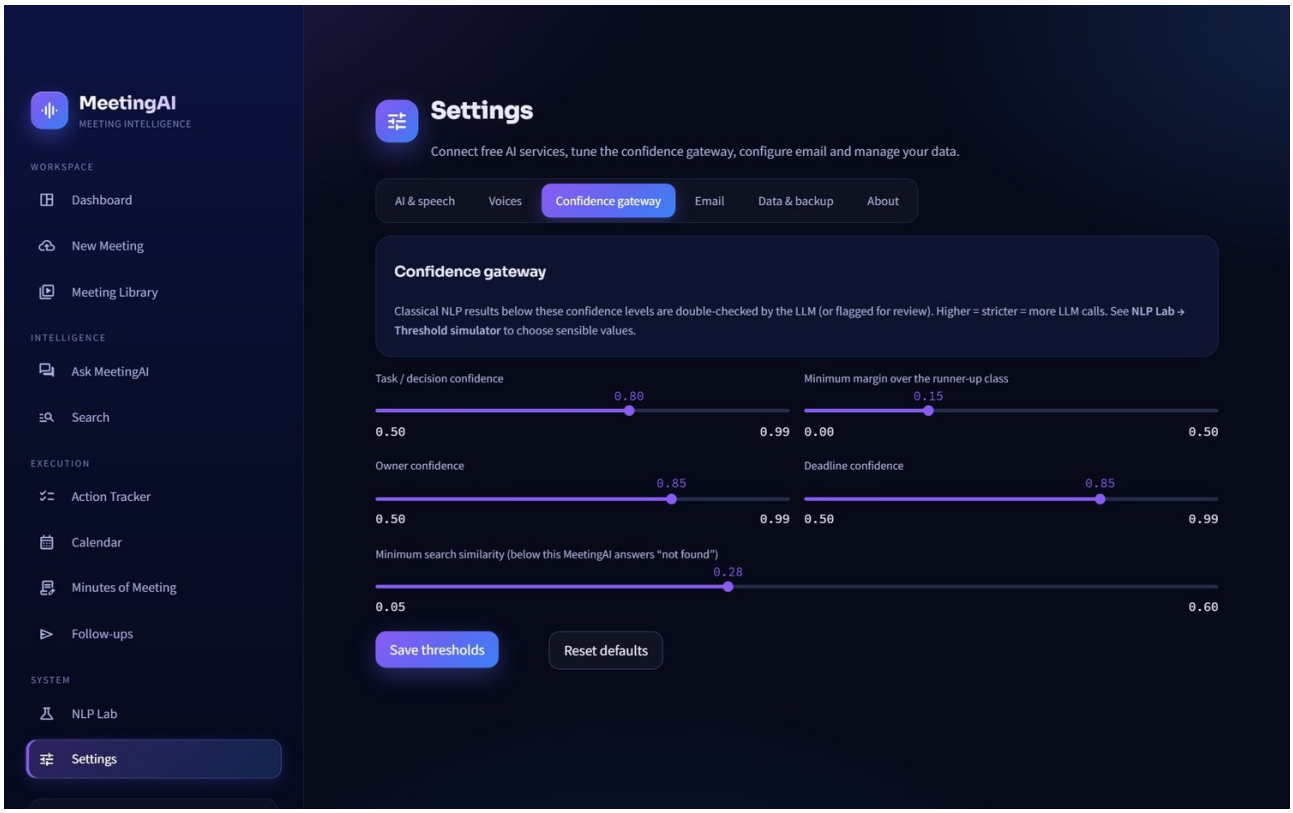
Connect free AI services, choose the speech and speaker engines, save team voices, tune the confidence gateway, configure e-mail, back up or restore data.



Settings - AI and speech



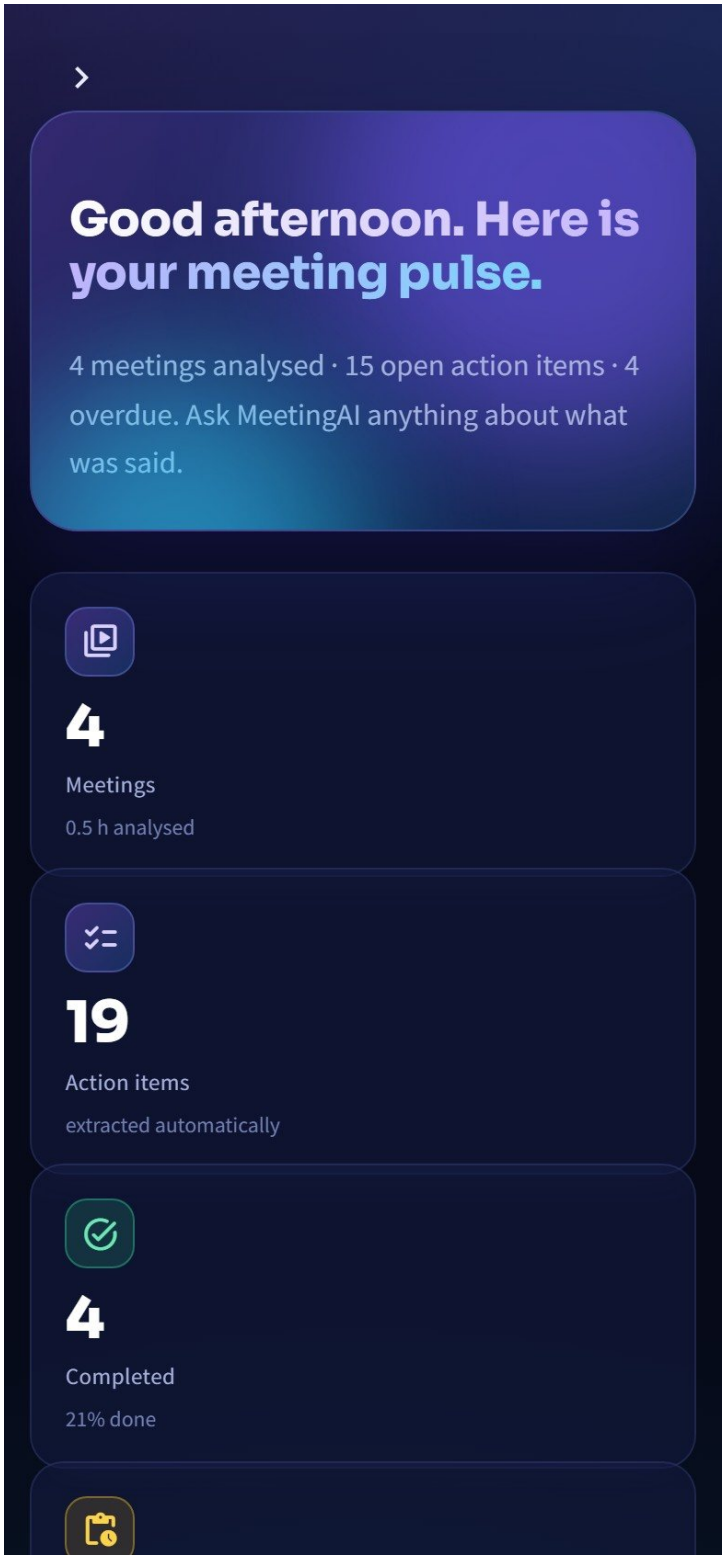
Settings - voice profiles



Settings - confidence gateway

3.14 Responsive design

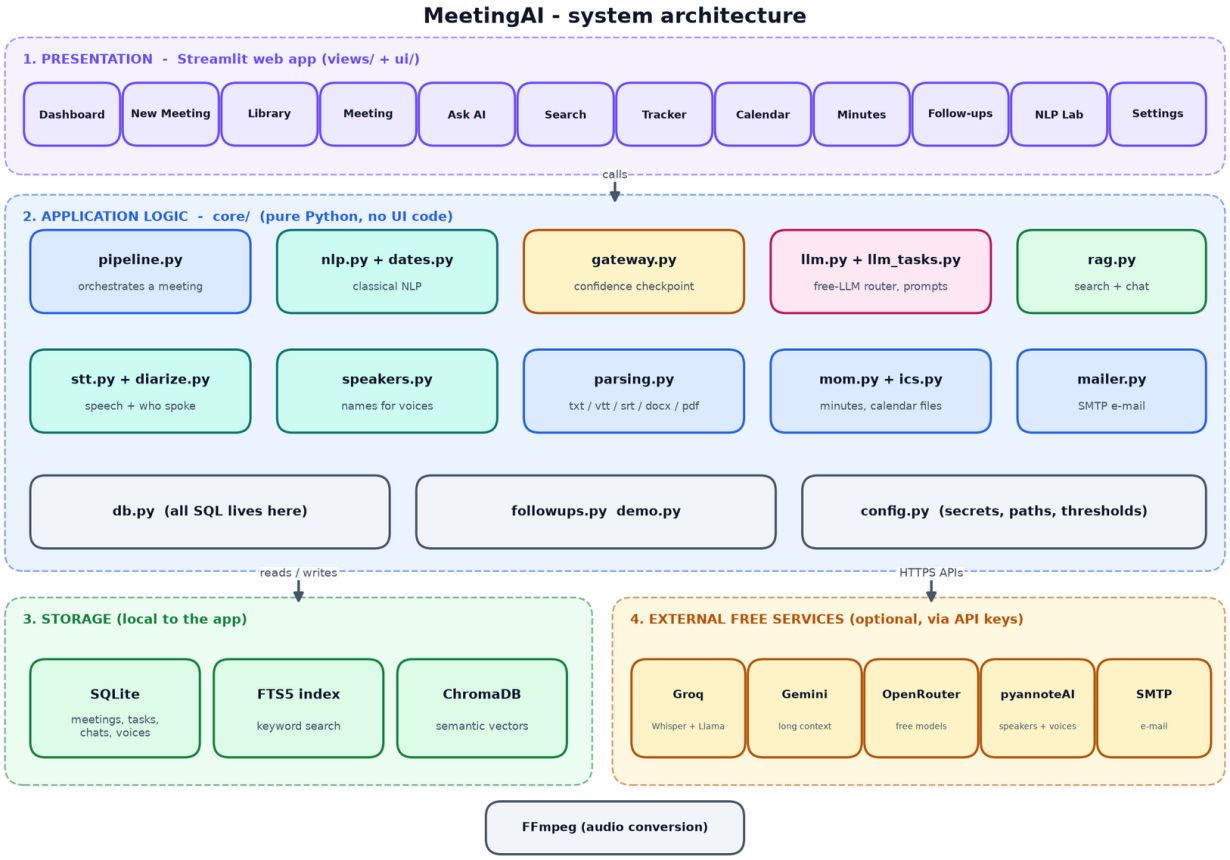
The layout adapts to phones: columns stack, the sidebar collapses after navigation, and tables scroll.



Phone layout

4. System architecture

MeetingAI has four layers. The strict rule is that **pages never contain business logic**: `views/` only draw and call `core/`; `core/` knows nothing about Streamlit pages.



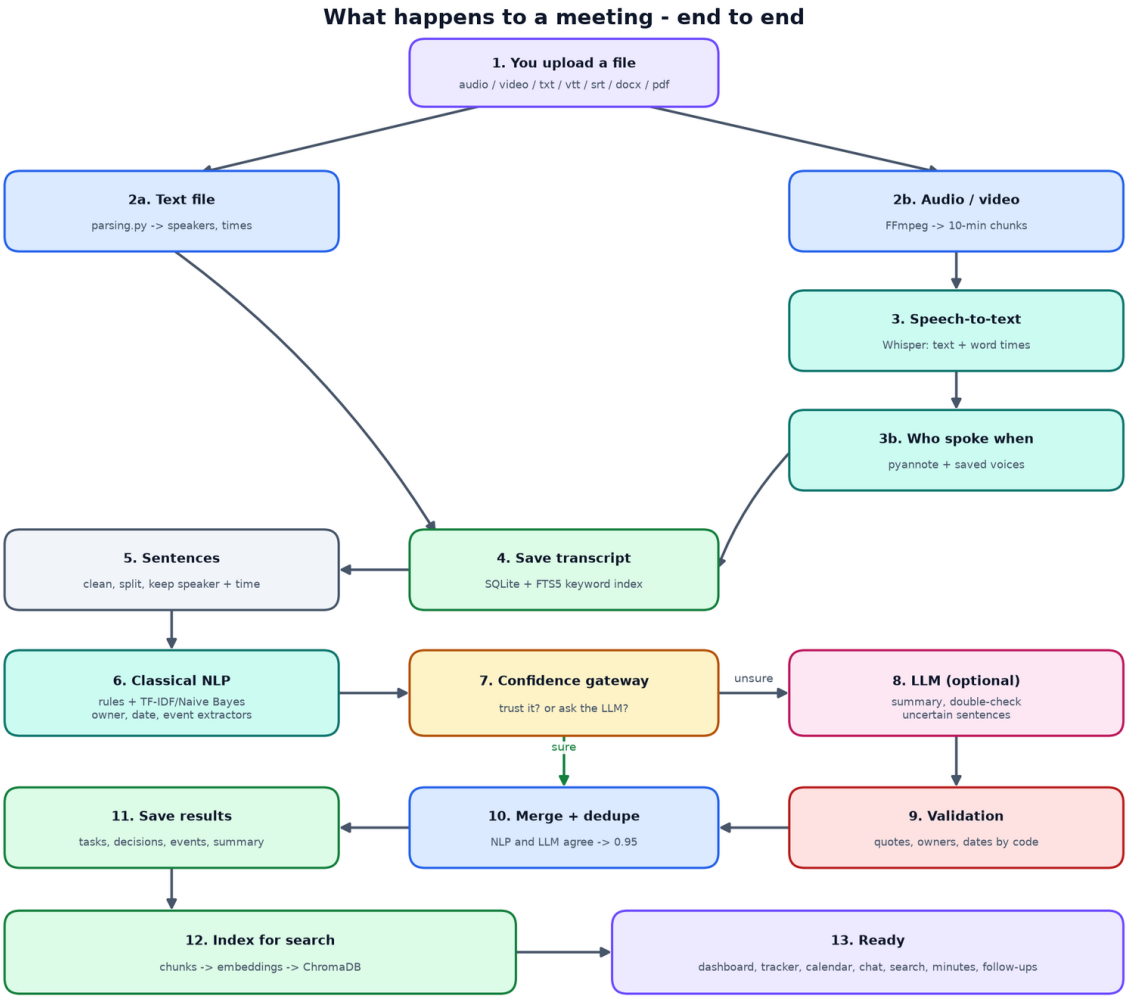
System architecture

Layer	Folder	Responsibility
Presentation	`views/`, `ui/`, `app.py`	Pages, theme, reusable HTML components, charts, navigation
Application logic	`core/`	Parsing, speech, speakers, NLP, gateway, LLM, search, minutes, e-mail
Storage	`data/` (created at run time)	SQLite database, FTS5 index, ChromaDB vectors, uploaded recordings
External services	via HTTPS	Groq, Gemini, OpenRouter, pyannoteAI, AssemblyAI, SMTP

Every external service is **optional**. With no keys at all, text transcripts still produce tasks, decisions, events, summaries, search and minutes using classical NLP only.

How a Streamlit app runs. Streamlit re-runs the whole script after every click. `app.py` therefore does the same four things each time: (1) apply the theme, (2) check the optional password, (3) draw the sidebar, (4) import and render the selected page. Per-user state (current page, selected meeting, chat session) lives in `st.session_state`; durable data lives in SQLite; expensive objects (the trained classifier) are cached.

5. End-to-end processing flow



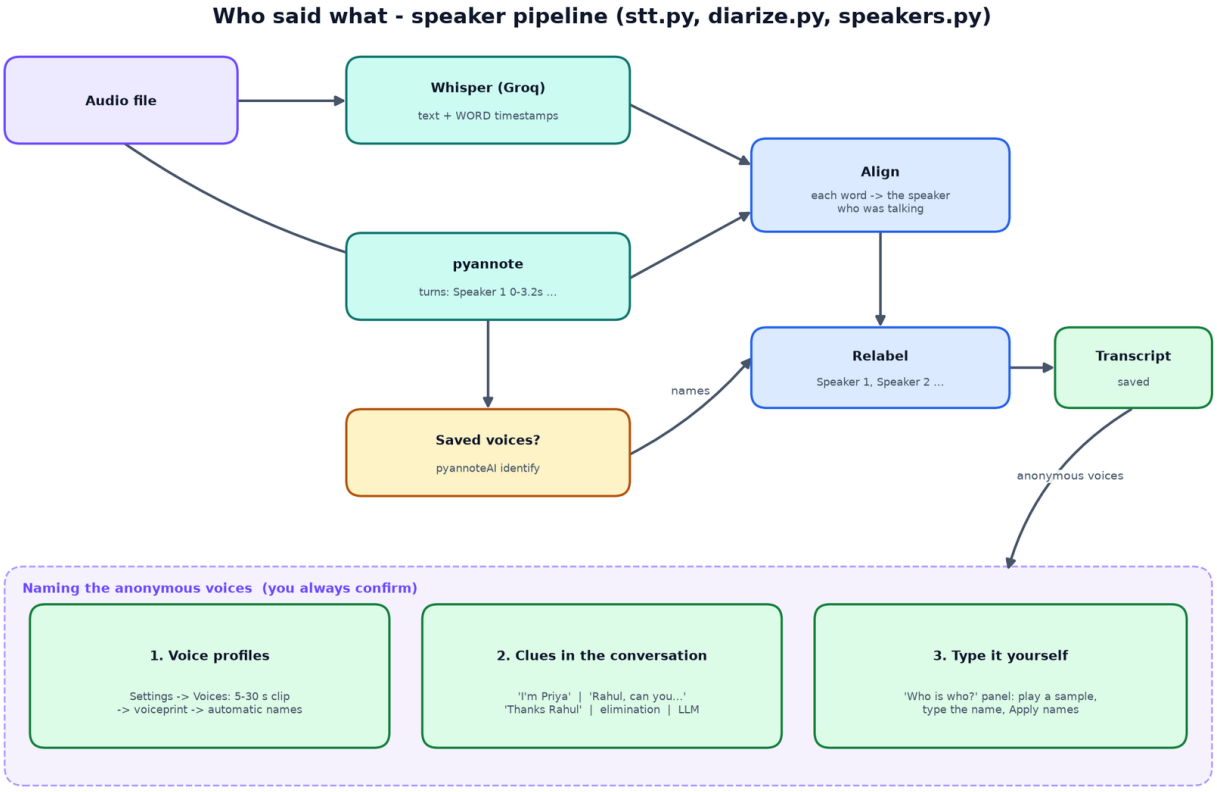
End-to-end flow

1. **Upload.** `views/upload.py` creates a meeting row and chooses the path by file type.
2. **Text files** go through `parsing.py`, which reads speaker lines (`[00:20] Rahul: ...`), subtitle files and header lines (`Meeting:`, `Date:`, `Attendees:`).
3. **Audio/video** is converted by FFmpeg into mono 16 kHz MP3 chunks of ten minutes, then transcribed (section 6).
4. The transcript is **saved** to SQLite and to the FTS5 keyword index.
5. `nlp.build_sentences` cleans the text (removes "um", "you know"), splits it into sentences and remembers speaker, timestamp and neighbouring sentences.
6. **Classical NLP** scores every sentence and extracts owner, deadline and events (section 7).
7. The **confidence gateway** decides which results are trusted (section 8).
8. If an LLM is configured it writes the summary, double-checks uncertain sentences and looks for items the rules missed (section 9).
9. **Validation** checks every LLM item against the transcript.
10. Results are **merged and de-duplicated**; when classical NLP and the LLM agree, confidence rises to 0.95.

11. Tasks, decisions, events, summary and the gateway log are **saved**.
12. Transcript chunks, decisions and tasks are **embedded and indexed** in ChromaDB.
13. The meeting becomes **ready**: dashboard, tracker, calendar, chat, search, minutes and follow-ups all use the new data.

6. Speech and "who said what"

Audio has no names in it. MeetingAI solves *who spoke when* with **pyannote**, then solves *what is this person called* in three complementary ways.



Speaker pipeline

6.1 Speech-to-text

`stt.py` converts media with FFmpeg and sends each chunk to **Groq Whisper (large-v3-turbo)**, asking for **word-level timestamps**. Chunks are time-shifted so the transcript has one continuous timeline. Whisper's 2-5 second fragments are merged into readable paragraphs, and segments Whisper probably hallucinated over silence are discarded. A local Whisper model (faster-whisper / openai-whisper) is used if installed and no cloud engine is configured.

6.2 Diarization with pyannote

`diarize.py` supports two ways to run pyannote:

	pyannoteAI hosted API	pyannote.audio locally
Needs	`PYANNOTEAI_API_KEY`	PyTorch (~2 GB), `HF_TOKEN`, accepted model terms
Works on free cloud hosting	Yes	No (memory)
Recognises saved voices by name	**Yes**	No (anonymous labels)

The hosted flow is: upload the audio to temporary storage (`media://...`), submit a **diarize** job (or an **identify** job when voice profiles exist), poll the job until it succeeds, and read the speaker **turns** (`Speaker 1: 0.0-3.2 s`, ...).

6.3 Aligning words with speakers

`assign_speakers` gives every Whisper word to the speaker whose turn contains the word's midpoint, so a sentence is split exactly where the voice changes. One-word interjections ("yeah") do not fragment a turn. If a recording has no word timestamps, the speaker with the largest time overlap wins the segment. Raw labels (`SPEAKER_00`) become `Speaker 1`, `Speaker 2` in order of first appearance.

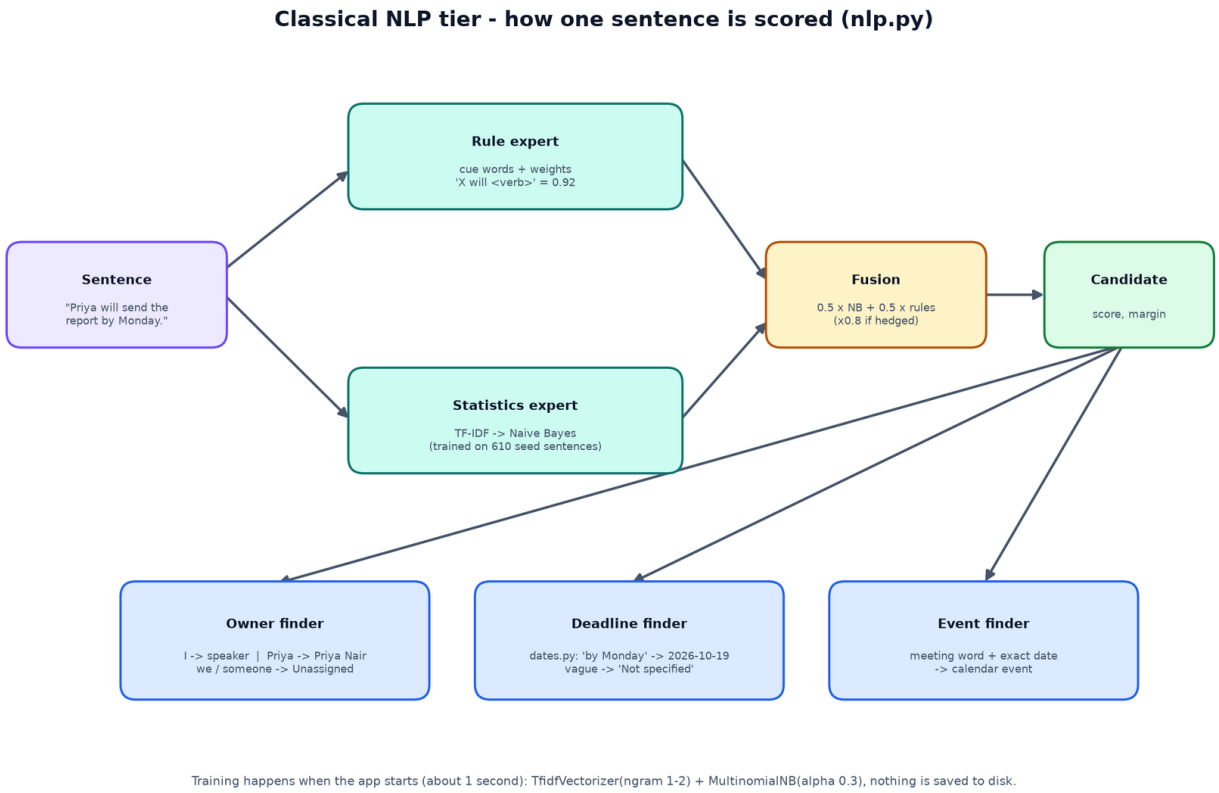
6.4 Putting names on voices

1. **Voice profiles (automatic).** Settings -> Voices: give a name and a 5-30 s clip of one person. pyannoteAI returns a **voiceprint** (a compact voice fingerprint) which is stored in the local database. On later recordings the *identify* job matches voices to voiceprints; a voice that matches well enough (default strictness 60/100) gets the name, the rest stay "Speaker N". Unknown people are never forced onto a name.
2. **Clues in the conversation** (`speakers.py`). Self-introductions ("I'm Priya"), being addressed ("Rahul, can you..." means the next voice is Rahul), being thanked ("Thanks Rahul" means the previous voice), elimination (one unnamed voice and one unused attendee) and an optional LLM that must quote the exact evidence. Names must be attendees or appear in the transcript.
3. **Manual.** The *Who is who?* panel with audio samples. Suggestions are never applied silently.

Applying names renames the transcript, task owners and decisions, updates the attendee list and refreshes the search index.

If the speaker service fails or has no credits, the transcript is still saved (as one speaker) with a visible warning.

7. Classical NLP: how sentences are understood



Classical NLP scoring

For each sentence two "experts" produce a score for four labels - **action, decision, question, discussion**.

The rule expert (`rule_scores`) looks for cue patterns with weights, combined as `1 - (1-w1)(1-w2)...`:

Cue	Weight
"action item", "next steps are"	0.95
"X will / 'll <action verb>"	0.92
"can you / could you <verb>"	0.88
"going to <verb>", "responsible for", "assigned to"	0.85-0.88
"needs to <verb>"	0.80
"we decided / agreed / resolved"	0.96
"the decision is", "Decision:"	0.96-1.0
"let's go with", "it's settled"	0.85-0.92

Hedging words (*maybe, might, perhaps, we could, consider...*) mark a sentence as a suggestion instead of a commitment. A question mark does not turn a request into a question: "Priya, can you send the report?" stays a task.

The statistics expert is a **TF-IDF + Multinomial Naive Bayes** classifier. It is trained every time the application starts (about one second) on **610 seed sentences** generated from templates (190 action, 110 decision, 100 question, 210 discussion); nothing is saved to disk. A logistic-regression model trained on the same data is shown in the NLP Lab only for comparison.

Fusion. `score = 0.5 x NaiveBayes + 0.5 x rules`, multiplied by 0.8 when the sentence is hedged. The margin is the gap to the best competing label.

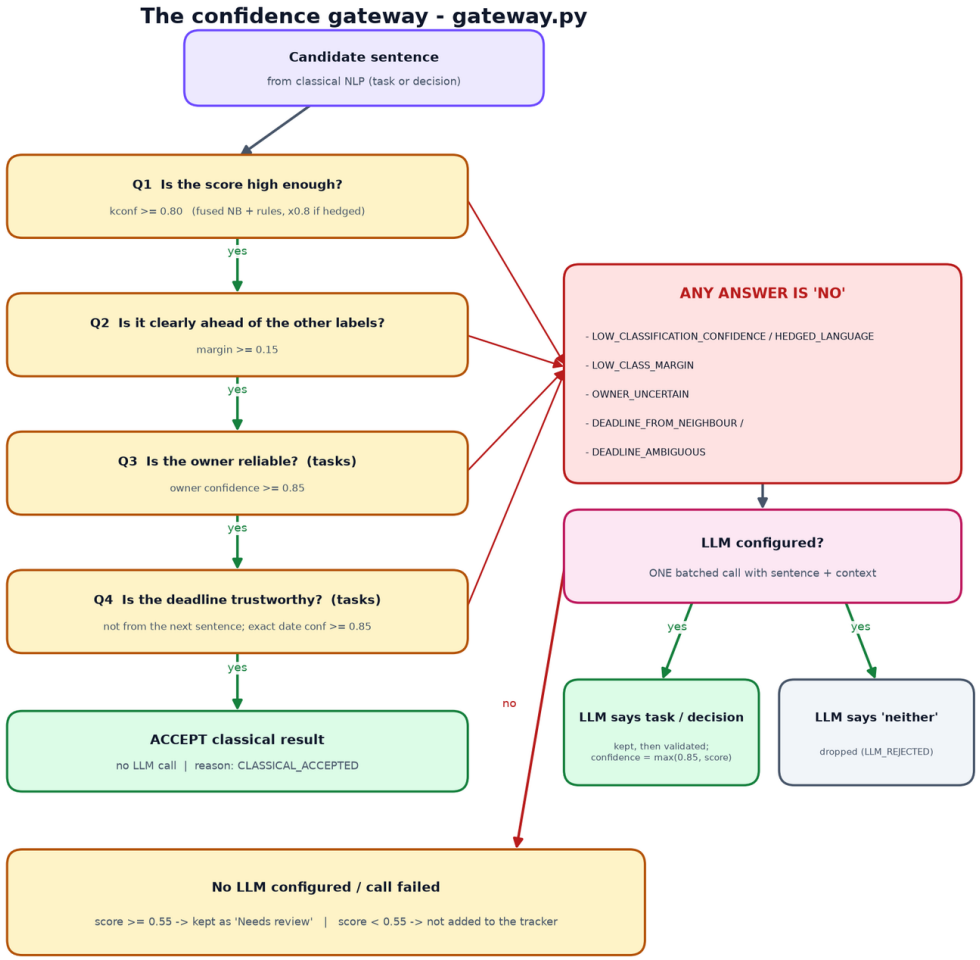
Around a candidate task three extractors run:

- **Owner** (`extract_owner`): "I" becomes the speaker; "Priya will..." matches a known speaker's full name ("Priya Nair"); "Priya, can you..." addresses Priya; "we / someone" means *Unassigned*; "he / she / they" cannot be resolved. Each result carries a confidence (0.95 known person ... 0.4 pronoun).
- **Deadline** (`dates.py`): finds expressions such as "by Monday", "next Friday at 3 PM", "end of the month", "in two weeks", "25th October" and converts them **relative to the meeting date**. Vague phrases ("next week", "soon") keep their text but get **no calendar date** - the deadline shows *Not specified*.
- **Event** (`extract_event`): a meeting-type word (call, demo, review, sync, workshop...) plus an exact date becomes a calendar event with an optional time.

The date resolver is plain, deterministic, unit-tested code. An LLM is never allowed to produce a calendar date.

8. The confidence gateway

The gateway (`gateway.py`) is the checkpoint between classical NLP and the LLM. Its four questions are asked in order; the first "no" sends the item onward.



Confidence gateway

Question	Compared	Default
Q1 Is the score high enough?	fused score (after hedge penalty)	>= 0.80
Q2 Is it clearly ahead of the other labels?	margin	>= 0.15
Q3 Is the owner reliable? (tasks)	owner confidence	>= 0.85
Q4 Is the deadline trustworthy? (tasks)	not from the neighbouring sentence; exact-date confidence	>= 0.85

Failing is not rejecting. It means "not sure". Then:

- **LLM configured:** the sentence and its context go to **one batched call**. If the LLM says task/decision it is kept (method *LLM fallback*, confidence at least 0.85) and validated; if it says "neither" the sentence is dropped.
- **No LLM:** an item with score >= 0.55 is kept and marked **Needs review**; below that it is not added to the tracker. Either way the decision is logged.

Every decision is stored with a **reason code** (`CLASSICAL_ACCEPTED`, `LOW_CLASSIFICATION_CONFIDENCE`, `LOW_CLASS_MARGIN`, `OWNER_UNCERTAIN`, `DEADLINE_FROM_NEIGHBOUR`, `HEDGED_LANGUAGE`, `LLM_CONFIRMED`, `LLM_REJECTED`, `MISSING_EVIDENCE`, ...) and a readable message. The thresholds are editable in Settings

and explored in the NLP Lab's *Threshold simulator*.

Worked examples

Sentence	Score	Owner	Date	Result
"Priya will send the project report by Monday."	0.98	0.95	0.95	accepted, no LLM
"Please book the conference room."	0.87	0.50	-	`OWNER_UNCERTAIN` -> LLM; else *Unassigned*
"Maybe we should consider hiring another developer."	low (hedged)	-	-	not a task
"We need to fix the login bug soon."	0.85	0.90 (we = nobody)	0.60 (vague)	accepted; owner *Unassigned*, deadline *Not specified*

9. The LLM layer and the validation layer

9.1 A router over free LLMs

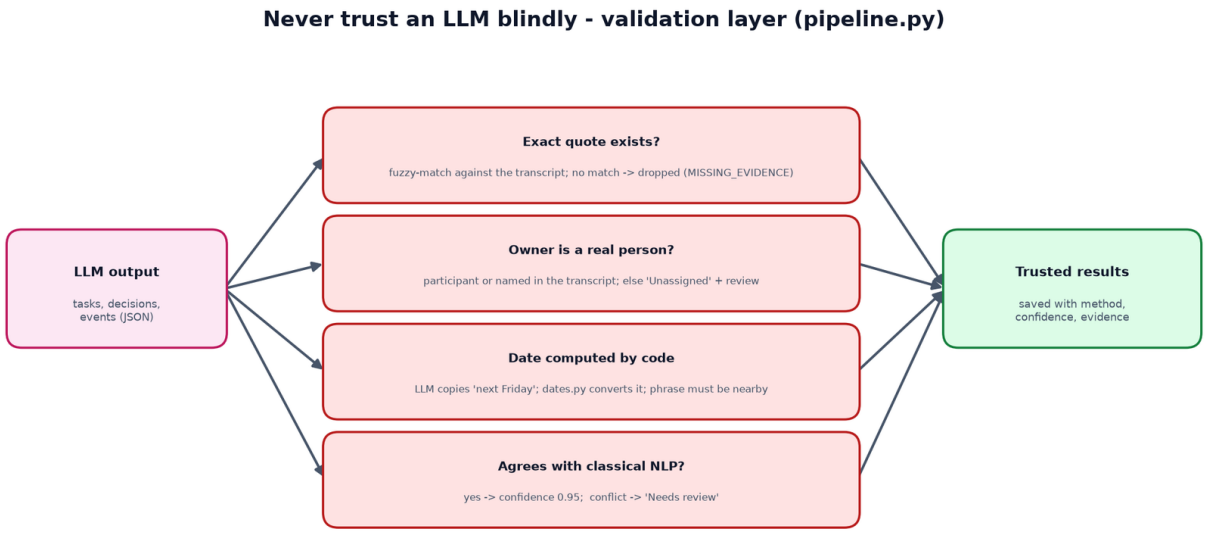
`llm.py` speaks the OpenAI-compatible API of **Groq**, **Gemini** and **OpenRouter** using plain HTTPS. Providers are tried in order; a rate limit puts a provider on a short cool-down and the next one is tried; for very long transcripts Gemini (large context window) goes first; if a free model has been retired the router asks the provider for its current model list and picks a replacement. Malformed JSON gets one automatic repair attempt.

9.2 What the LLM is asked to do

Call	Purpose
Analysis (per chunk of transcript)	Summary, key points, topics, conclusions, plus independent task/decision/event suggestions
Merge (long meetings)	Combine chunk summaries into one
Adjudication (one batch)	"Is each uncertain sentence a task, a decision or neither?"
Answer	Grounded chat/search answers (section 10)
Naming, polishing (optional)	Speaker-name evidence, friendlier follow-up wording

The prompts demand **verbatim quotes** and **copied date phrases** instead of computed dates.

9.3 Validation: never trust an LLM blindly

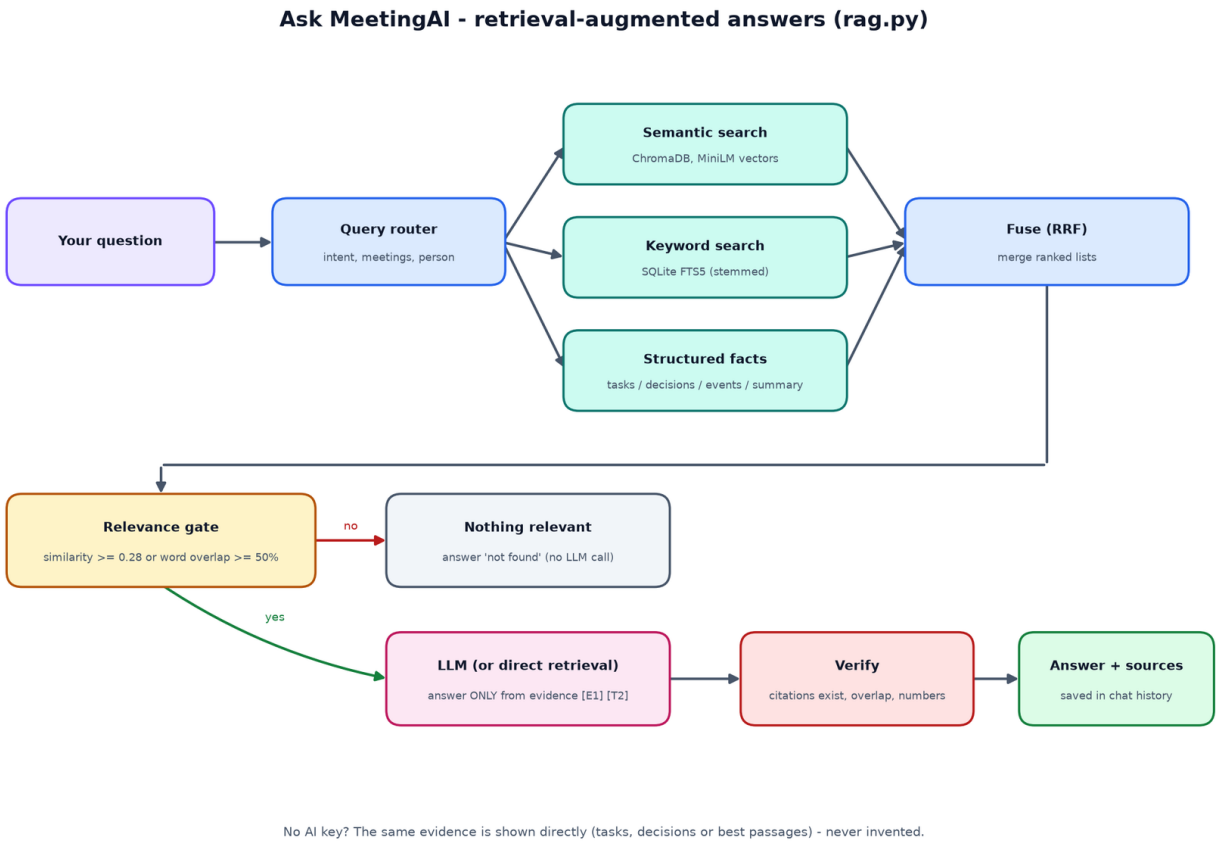


Validation layer

- **Quote check.** Each task/decision/event must include an exact quote; fuzzy matching finds it in the transcript or the item is dropped (`MISSING_EVIDENCE`).
- **Owner check.** The owner must be a participant or appear in the transcript; otherwise *Unassigned* + review.
- **Date check.** Our own resolver converts the copied phrase, and the phrase must actually occur near the quote.

- **Agreement.** An item found by both classical NLP and the LLM is marked *NLP + LLM agree* (confidence 0.95). Different owners produce `CONFLICTING_EXTRACTIONS` and a review flag.

10. Search and the chatbot



Retrieval-augmented answers

Indexing. The transcript is cut into chunks of about 110 words, each embedded by the **all-MiniLM-L6-v2** model (ONNX, 384 numbers per chunk, no PyTorch) and stored in a **ChromaDB** collection (cosine distance). Decisions and tasks are indexed too. SQLite is the source of truth - ChromaDB can always be rebuilt from it.

The query router (`route_query`) reads the question and decides which sources matter:

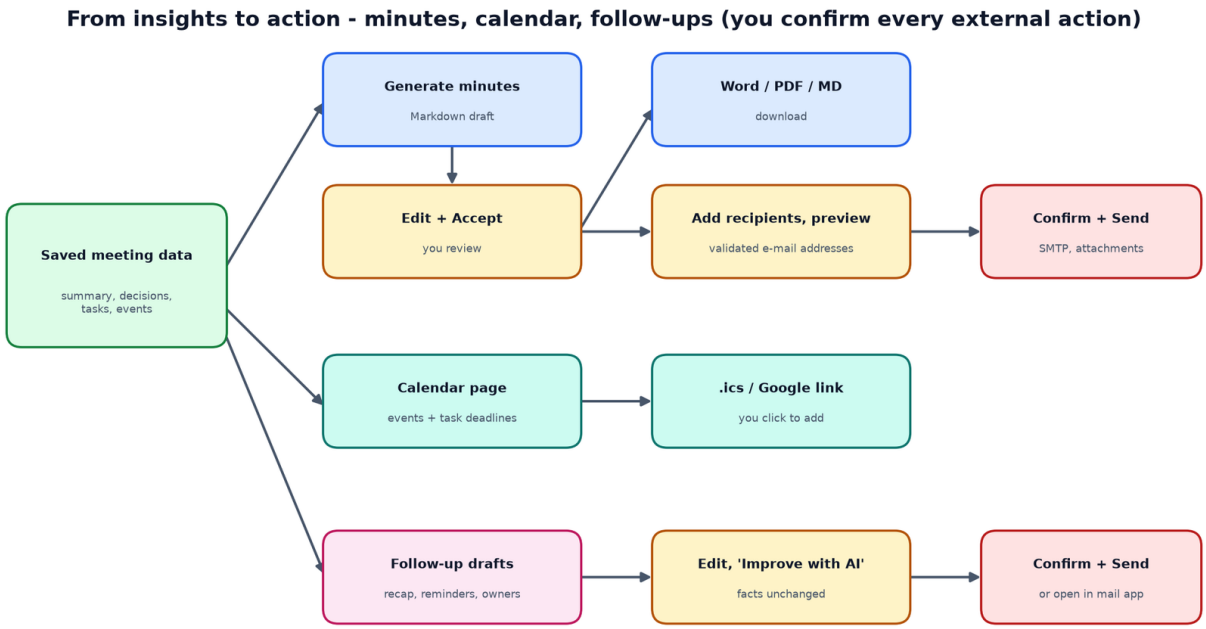
Question looks like	Uses
"who is responsible...", "deadlines", "pending tasks", a person's name	task table (+ passages)
"what did we decide..."	decisions table
"upcoming events", "schedule"	events table
"summarise...", "compare across..."	saved summaries
"last meeting", "last three meetings", "on October 10", a meeting name	limits the search to those meetings
text in quotes	exact-phrase search
anything else	hybrid search

Hybrid search. Semantic search (meaning) and SQLite FTS5 keyword search (words, with stemming) each return a ranked list; **Reciprocal Rank Fusion** merges them so passages that rank well in both rise to the top.

Relevance gate. If nothing is relevant (similarity >= 0.28, or at least half of the question's content words appear in the text) the answer is "not found" and **no LLM is called**.

Answering. The best evidence is labelled `[E1]`, `[T2]`, `[D1]` ... and given to the LLM with strict rules (answer only from this, cite it, state what is missing). Afterwards the code verifies that cited ids exist, measures how much of the answer is supported by the cited text and warns about numbers that are not in the evidence. Without an LLM the same evidence is shown directly (a "direct retrieval" answer). Questions, answers, sources and routing details are saved in the chat history.

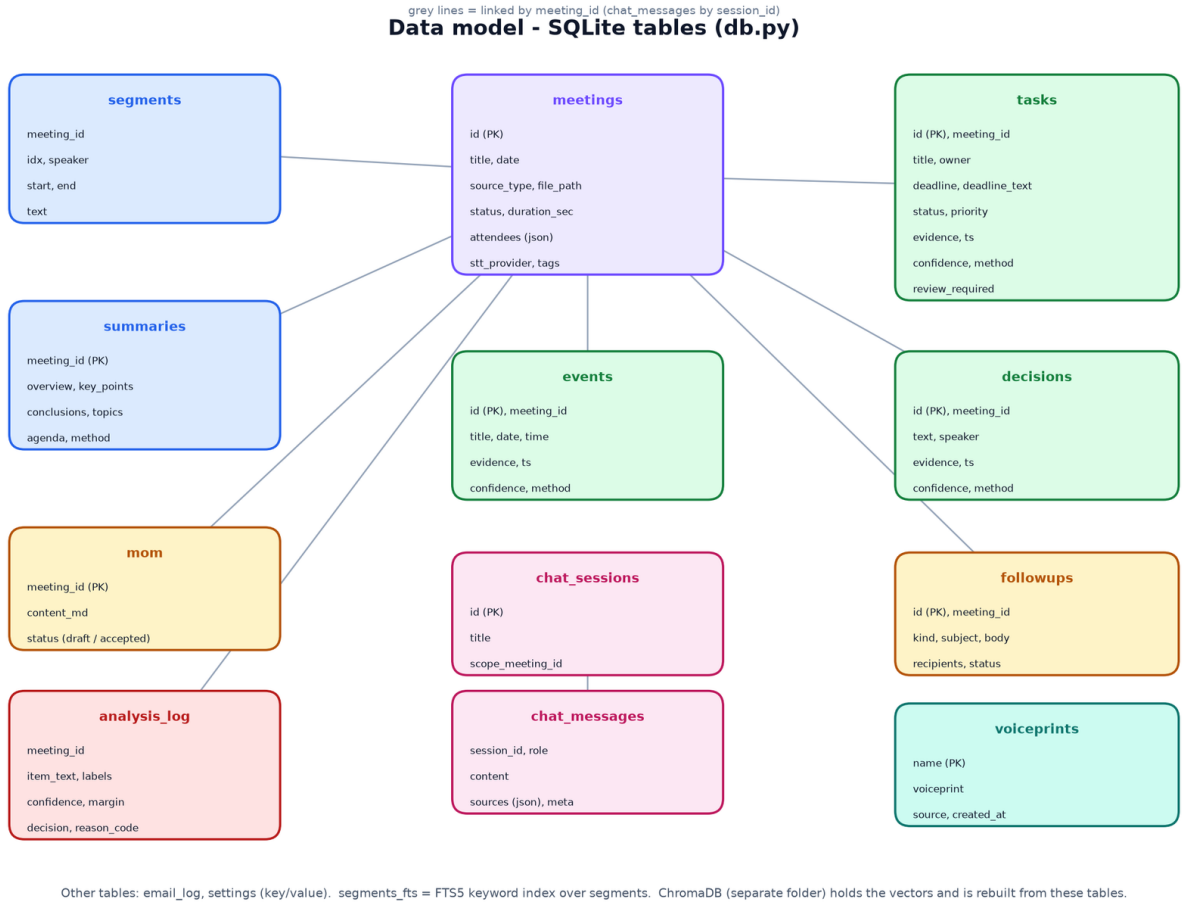
11. Minutes, calendar, follow-ups and e-mail



From insights to action

- **Minutes** (`mom.py`) are generated *on demand* from saved data (details, attendees, agenda, summary, highlights, decisions, action items, events, conclusions). Uncertain rows carry a warning mark. The editor accepts Markdown; **Accept** unlocks e-mailing; exports are Word, PDF and Markdown, produced by our own renderer (python-docx / ReportLab).
- **E-mail** (`mailer.py`) uses SMTP (Gmail app password, Outlook, Yahoo, Brevo ...). Addresses are validated, a preview is shown, a confirmation tick is required, and the attachments (Word and/or PDF) are chosen by the user. A "Connect your e-mail" panel appears wherever sending is needed and SMTP is not configured; if sending is impossible a `mailto:` link is offered instead.
- **Calendar** (`views/calendar.py`, `ics.py`) merges events and task deadlines. Items can be exported as `.ics` or added through a Google Calendar link - the user confirms by clicking, so no account permissions are needed.
- **Follow-ups** (`followups.py`) are drafted from decisions, pending tasks per person, unowned tasks and upcoming events. They are **never sent automatically**.

12. Data model and storage



Data model

SQLite (`data/meetingai.db`) holds every durable fact: meetings, transcript segments (plus the `segments_fts` keyword index), summaries, decisions, tasks, events, minutes, chat sessions and messages, follow-ups, the gateway log (`analysis_log`), e-mail log, settings and **voiceprints**. ChromaDB (`data/chroma/`) holds only vectors, and uploaded recordings live in `data/uploads/`.

On free cloud hosting the disk is temporary, so **Settings -> Data & backup** creates a zip (database + vectors + recordings) that can be restored later.

13. User-interface design

The interface is built entirely with Streamlit plus ~240 lines of custom CSS (`ui/theme.py`):

- **Palette:** deep navy (#060A19) background with violet (#8B5CF6) to blue (#3B82F6) gradients, glass-style cards, and green / amber / red reserved for status.
- **Typography:** Sora for headings, Inter for text, JetBrains Mono for timestamps; Material Symbols icons.
- **Components** (`ui/components.py`): metric cards, status chips, speaker avatars, transcript viewer, month grid, steppers, empty states.
- **Behaviour:** smooth fade-ins, animated hero, loading steppers and spinners, toast messages, helpful empty states, responsive columns, the sidebar collapses after navigation on phones.
- **Consistency:** every page uses the same header, section titles, cards and buttons.

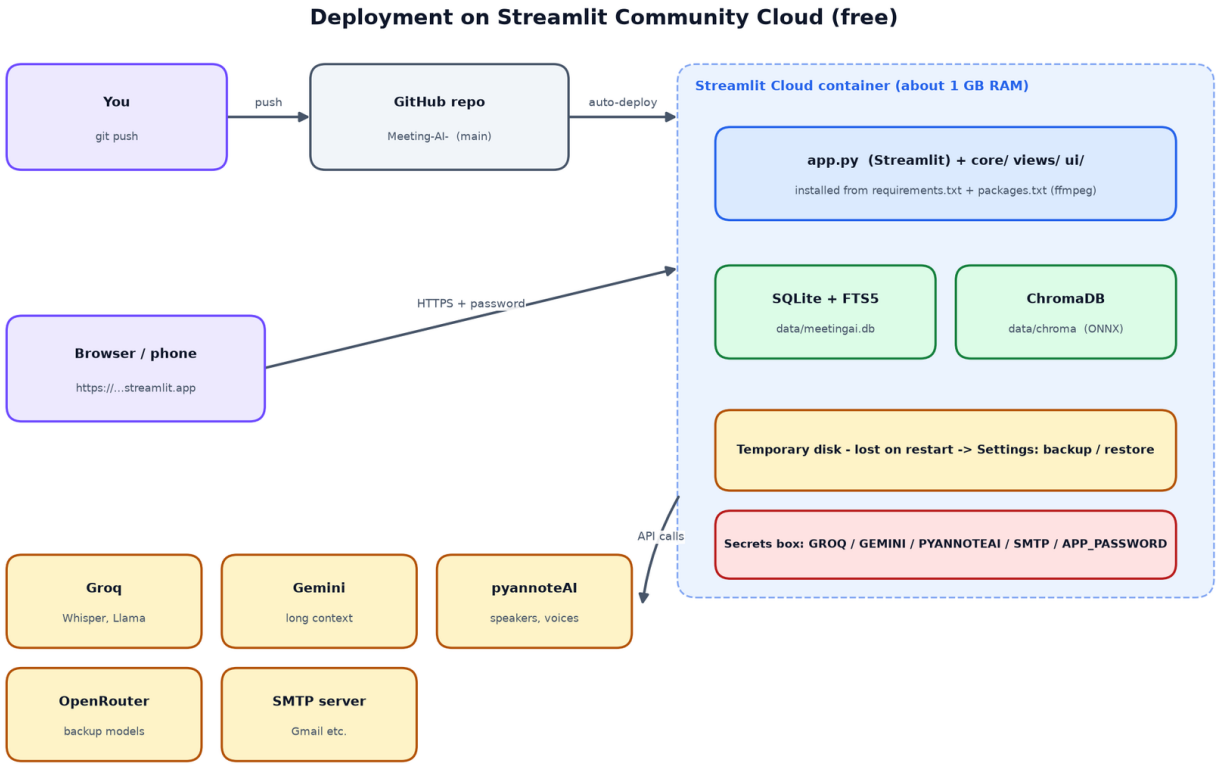
14. Technology stack and why each piece was chosen

Job	Technology	Why
Interface	**Streamlit** + custom CSS	One language (Python), fast to build, free hosting
Charts	Plotly	Interactive and themeable
Structured data	**SQLite**	A full database in one file; nothing to install or pay for
Keyword search	**SQLite FTS5** (porter stemming)	Built in and fast
Semantic search	**ChromaDB** + **ONNX MiniLM**	Local vector database; small model without PyTorch fits free hosting
Classical NLP	**scikit-learn** (TF-IDF, Naive Bayes), regular expressions, **rapidfuzz**	Fast, offline, explainable, produces confidence
Dates	Custom resolver (`dates.py`)	Deterministic and testable; never hallucinates
Speech-to-text	**Whisper** via Groq (or local)	Best open ASR; Groq's free tier is fast
Speakers	**pyannote** (pyannoteAI hosted or local); AssemblyAI alternative	Leading diarization toolkit; hosted API fits free hosting and recognises voices
LLMs	**Groq (Llama)**, **Gemini**, **OpenRouter free models**	All free tiers; automatic fallback when one is limited
Audio	**FFmpeg**	The standard tool
Documents	**python-docx**, **ReportLab**	Pure Python, no Office needed
E-mail	**SMTP**	Works with any provider
Testing	**pytest**, Streamlit AppTest, fake HTTP servers	Fast, no real accounts required
Deployment	**Streamlit Community Cloud** + GitHub	Free, automatic redeploy on push

15. Security, privacy and honesty by design

- **Keys** are never written to the database or repository: Streamlit secrets, environment variables, or typed into Settings (session only).
- **Password gate:** set `APP_PASSWORD` to protect a public deployment.
- **Confirmation before side effects:** e-mails and follow-ups are sent only after an explicit tick/click; calendar additions are links the user clicks.
- **Untrusted content:** transcripts and LLM output are treated as data - they cannot trigger actions, and LLM output must pass validation.
- **What leaves the machine** (only when configured): audio to Groq and pyannoteAI (or AssemblyAI); transcript text to Groq / Gemini / OpenRouter. The "use AI model" switch can turn LLM use off per meeting.
- **Voiceprints** stay in your own database; create them only for people who agreed, and delete them any time.
- **Honesty:** unknown owners show *Unassigned*, unknown dates *Not specified*, unanswerable questions *not found*, uncertain items *Needs review*.

16. Deployment



Deployment architecture

The application is deployed on **Streamlit Community Cloud** straight from GitHub:

1 Code is pushed to https://github.com/nidhitiwari23/Meeting-AI-.

2. Streamlit Cloud installs `requirements.txt` (Python packages) and `packages.txt` (FFmpeg) and runs `app.py`.
3. API keys are entered once in the app's **Secrets** box (`GROQ_API_KEY`, `GEMINI_API_KEY`, `PYANNOTEAI_API_KEY`, `SMTP_*`, `APP_PASSWORD`).
4. Every later `git push` redeploys automatically.

Live application: https://gd4n8x8sjkkxmdhfptkc9m.streamlit.app/

Things to know about the free tier: about 1 GB of memory (the design avoids PyTorch for this reason), the app sleeps when idle, and the disk is temporary - use the backup feature. Alternatives: Hugging Face Spaces (more RAM), Render, Google Cloud Run, or run locally.

17. Testing and evaluation

79 automated tests (`python -m pytest tests`) cover:

Area	Examples
Dates	weekdays, "next Friday", "end of month", vague phrases get no date, ambiguous `03/04` not guessed
Classical NLP	owner from "I'll", full-name matching, "Priya, can you...", missing owner stays *Unassigned*, hedged sentences, decisions, events
Parsing	speaker lines, timestamps, headers, VTT/SRT, empty files
Pipeline	the three sample meetings end to end; the **LLM path with a fake LLM** proving invented quotes and invented owners are rejected
Search & chat	keyword and semantic search, "not found" for unrelated questions, owners in answers
LLM router	fall-through on rate limits, model auto-discovery, JSON repair, all-providers-down
Speakers	alignment of words to turns, hosted-API round trip against a fake pyannoteAI server (upload, diarize, identify, voiceprint), voice-sample cutting with FFmpeg, name suggestions, applying names
Output	Word/PDF/HTML export, SMTP against a local mail server, `.ics` files, audio chunking
UI	every page renders without errors (Streamlit AppTest)

Classifier evaluation (NLP Lab). On 42 hand-written sentences never used for training: rules ~98%, Naive Bayes ~76%, logistic regression ~81%, fused model ~93% accuracy. Both sets were written by the project authors, so these numbers show that the pipeline works, not how it will perform on real company meetings - which is exactly why the confidence gateway and the LLM safety net exist.

18. Limitations and future work

Limitations

- Live calls to Groq, Gemini, OpenRouter, AssemblyAI and pyannoteAI, and real e-mail delivery, require your own keys; during development these paths were verified against fake servers. Use *Settings -> Test connections* after adding keys.
- Speaker naming accuracy depends on audio quality; overlapping speech and very short interjections can be mislabelled; voice recognition needs saved voice profiles.
- Free tiers have rate limits and model names change (the router adapts automatically).
- The classical tier is a baseline trained on a small seed set; English is the main supported language.
- The free host's disk is temporary (backup/restore provided).

Future work: train on real annotated meetings and calibrate probabilities; multi-language support; direct Google Calendar creation after sign-in; user accounts and per-user data; a persistent hosted database; live (real-time) meeting capture.

19. Source-code map (one line per file)

Entry point and configuration

File	What it does
`app.py`	Page setup, optional password gate, sidebar, loads the selected page
`requirements.txt` / `packages.txt`	Python packages / system packages (FFmpeg) for deployment
`requirements-local.txt`	Optional: local pyannote.audio (PyTorch)
`.streamlit/config.toml`, `secrets.toml.example`	Dark theme and upload limit / template for API keys

`core/` - the application logic

File	What it does
`config.py`	Paths, defaults, secret lookup (session > secrets > environment)
`db.py`	All SQLite tables and every read/write function, FTS5 search
`parsing.py`	Reads txt / vtt / srt / docx / pdf into speaker segments
`stt.py`	FFmpeg conversion, Whisper (Groq/local), AssemblyAI, merging, orchestration
`diarize.py`	pyannote (hosted/local), voiceprints, word-to-speaker alignment
`speakers.py`	Name suggestions for anonymous voices, applying names
`dates.py`	Finds and converts date/time expressions
`seed_data.py`	Training and test sentences for the classifier
`nlp.py`	Sentences, rules, Naive Bayes, owner/deadline/event extraction, offline summary
`gateway.py`	The confidence gateway and reason codes
`llm.py`	Free-LLM router with fallback and model discovery
`llm_tasks.py`	Prompts and parsers for everything the LLM does
`pipeline.py`	Orchestrates analysis, validation, merging and saving
`rag.py`	ChromaDB index, hybrid search, query router, grounded answers
`mom.py`	Minutes generation and Markdown/HTML/Word/PDF rendering
`mailer.py`	SMTP sending, address validation, `mailto:` links
`ics.py`	Calendar files and Google Calendar links
`followups.py`	Drafts follow-up messages and reminders
`demo.py`	Three sample meetings

`ui/` and `views/` - the interface

File	What it does
`ui/theme.py`	All CSS (colours, cards, animations, responsive rules)
`ui/components.py`	Reusable HTML: cards, chips, transcript viewer, calendar grid
`ui/charts.py`, `ui/nav.py`	Themed Plotly charts; navigation helpers
`views/dashboard.py` ... `views/settings.py`	One file per page (dashboard, upload, library, meeting, chat, search, tracker, calendar, mom, followups, nlplab, settings)
`views/_tasks.py`, `_speakers.py`, `_email.py`	Shared panels: editable task table, *Who is who?*, *Connect e-mail*

Tests and docs: `tests/` (79 tests), `docs/` (this report, the code walkthrough, the project guide, diagram and screenshot tools).

The *Code Walkthrough* document explains every file in depth.

20. Glossary

Action item - a task someone agreed to do. **ASR** - automatic speech recognition. **Chunk** - a short piece of transcript stored for searching. **Confidence** - a number from 0 to 1 showing how sure a model is. **Diarization** - working out who spoke when. **Embedding** - a list of numbers representing the meaning of a text. **FTS5** - SQLite's built-in full-text search. **Gateway** - the checkpoint deciding whether classical NLP is trusted or the LLM must double-check. **Hallucination** - confident but false AI output. **LLM** - large language model (Llama, Gemini). **Margin** - gap between the best and second-best guess. **MoM** - Minutes of Meeting. **Naive Bayes** - a simple probability-based classifier. **NLP** - natural language processing. **RAG** - retrieval-augmented generation: find relevant text first, then answer only from it. **RRF** - reciprocal rank fusion, a way to merge two ranked lists. **TF-IDF** - a score of how important a word is in a text compared with all others. **Voiceprint** - a compact numeric fingerprint of a voice. **Whisper** - OpenAI's open speech-recognition model.